

GREYNOC OPERATOR REFERENCE

Operator Handbook

Doctrine and readiness, the operator platform, defensive scripting, detection and incident response, and the AI and post-quantum frontier - with the shared conventions the playbook library runs on.

DOCUMENT

GN-PUB-OPHB-001

PUBLICATION DATE

2026-09-20

SOURCE

GreyNOC/Playbooks

VERSION

v1.0.0

CLASSIFICATION

PUBLIC // GreyNOC Field Reference

DISCIPLINE

Authorized, evidence-backed operations

Authorization first. Least impact. No fabrication. Every procedure here applies only to systems you own or are explicitly, in writing, permitted to administer, monitor, assess, or test, and every finding must be independently reproducible from evidence. This handbook is the practice around the playbook library, not a replacement for it: cross-references use the three-collection notation in CONVENTIONS Section 5 (D&R-NN , PB-NN , TX-NN). MITRE mappings are pinned to **ATT&CK Enterprise v19.2** and **ATLAS collection 2026.06**; re-map to the collections your platform carries (CONVENTIONS Section 4). Standards, platform settings, and framework identifiers move - re-verify before relying on any of them.

PUBLICATION MAP

Contents

01 GreyNOC Operator Handbook

02 CONVENTIONS

SOURCE: OPERATOR-HANDBOOK.md

GreyNOC Operator Handbook

What this is. One reference for the whole GreyNOC operator job: the doctrine that governs authorized work, the platforms an operator builds and hardens, the scripting craft that turns manual work into repeatable defensive product, the operational sequence for detection and incident response, and the frontier domains - AI-assisted operations and the post-quantum transition - that now sit inside ordinary defensive work rather than beside it.

Who it is for. SOC and NOC operators, detection engineers, incident responders, system administrators, and authorized offensive operators working under signed scope. It assumes command-line comfort and no more.

What it is not. It is not a replacement for the playbook library. The playbooks carry the detection logic, the defect-class catalogs, and the technique-level mappings; this handbook carries the practice around them and points at the right playbook at each decision. Cross-references use the three-collection notation from CONVENTIONS Section 5: D&R-NN (Detection & Response), PB-NN (AI · Post-Quantum · E2EE), TX-NN (Transmission & Physical Layer). A bare number is never a valid citation across collections.

Three standing rules. They are not negotiable, and nothing in this handbook overrides them.

1. **Authorization first.** Act only on systems you own or are explicitly, in writing, permitted to administer, monitor, assess, or test. The scope document is the boundary, not your assumptions.
2. **Reproducible or it didn't happen.** Every finding, indicator, metric, and report artifact must be independently reproducible from the evidence. If you cannot reproduce it, it is not a finding.
3. **Least impact.** Prefer read-only observation. Prove the point with the smallest footprint that proves it, then stop.

Part I - Doctrine and Readiness

1. How to use this handbook

Read Part I once, in full, before operating. It defines the boundary and the evidence standard that the rest of the document assumes.

After that, the handbook is reference material. Five entry points cover most use:

You are...	Start at	Then
Setting up a new operator	Section 6-Section 10 (platform)	Section 42 readiness checklist
Writing or reviewing a script	Section 11-Section 13, Section 17	Section 21 catalog, Section 42 code checklist
Working a live incident	Section 29 (first hour)	Section 30 or Section 31 by platform, then Section 32
Standing up a detection	Section 27	The matching D&R-NN playbook
Preparing authorized testing	Section 2, Section 39	CONVENTIONS Section 6-Section 8

Versioned claims. Standards, platform settings, and framework identifiers move. Every claim in this handbook that depends on an external version states the version and the date it was checked. Where a section cites MITRE, the baseline is ATT&CK Enterprise v19.2 and ATLAS collection 2026.06, verified 2026-08-09, per CONVENTIONS Section 4. Re-verify before you rely on any of it.

Example data. Network examples use RFC 5737 (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and RFC 3849 (2001:db8::/32) documentation space. Identities, domains, and tokens in examples are illustrative and non-attributable.

2. Authorization, scope, and the boundary

Authorization is the single thing that separates defensive security work from a crime. It is a document, not a feeling, and not a verbal "go ahead" in a hallway.

What counts as authorization.

Context	Sufficient authorization
Your own hardware and accounts	Ownership
Employer systems	Written internal authorization naming systems, activity, and window
Client engagement	Signed SOW or rules of engagement naming assets, methods, and dates
Bug bounty or VDP	An in-scope, in-policy program page, read in full, plus CONVENTIONS Section 6
Exercise (red, blue, purple)	Signed scope, deconfliction channel, and a named White cell

A screenshot of an email approval is not sufficient for professional engagements. Use a signed scope-of-work or rules-of-engagement document, and keep it reachable in the field.

Scope is a hard boundary. Map in-scope and out-of-scope assets explicitly before you start, and re-check each session - programs and engagements change their scope over time. "I was only looking" is not a defense if the asset was out of scope.

Four questions before any action that changes state. Write the answers down.

1. Who owns this asset, and what document authorizes me to touch it?
2. What is the objective, stated in one sentence, with a definition of done?
3. What data will I handle, and what is its sensitivity?
4. What is the rollback, and who approves it?

Out of scope for GreyNOC operators, always. Destructive payloads and denial of service intended to cause real harm; mass or indiscriminate targeting; malware for unauthorized use; supply-chain compromise; and detection-evasion built for real-world attack rather than research or defense. These are excluded regardless of who asks.

Stop conditions. Stop, preserve, and escalate on any of the following, in any activity:

- Evidence of a prior real-world compromise you did not cause.
- Customer or third-party personal data at rest that you did not expect to encounter.
- Anything outside the authorized boundary, including a pivot that would cross it.
- Evidence another party is exploiting the same weakness concurrently.
- A finding that cannot be proven without causing genuine harm.

Stopping is a disciplined action with a record, not an abandonment. Note the time, what you saw, what you did not do, and who you told.

3. The evidence standard

No fabrication. This is the house rule, and it binds every artifact GreyNOC produces: detections, findings, indicators, metrics, reports, and dashboards. Concretely:

- Every claim traces to an observation, with the command or query that produces it.
- No speculative severity. Severity follows what you proved, not what you suspect.
- No invented impact. If you did not demonstrate the consequence, describe the control failure and say the consequence is not demonstrated.
- No copied advisory text passed off as original analysis.
- Absence of evidence is reported as absence of evidence, never as absence of the thing.

Telemetry sufficiency comes before conclusions. An empty alert queue for a medium you do not instrument is a coverage gap, not a clean result. Before you trust the absence of a signal, confirm you would have seen it: the log source exists, it is enabled, its retention covers the window, and the rule that would fire is deployed and not suppressed. TX-01 and TX-02 state this with unusual force for physical media, where passive interception is undetectable in principle and the control is cryptographic rather than telemetric.

Reproducibility for nondeterministic systems. Where a result is stochastic - model output, race-dependent behavior, timing-sensitive conditions - one success is not a finding. Record the trial ledger: the system and version, the parameters, the trial count, the success count, and the interval. The GreyNOC bar and its notation live in CONVENTIONS Section 7.6 and Section 8.3; the canonical short form is `successes/trials @ parameters (95% Wilson CI low-high)`, and `2/50` is reported as `2/50`, never as "reproducible."

Evidence grading. When a claim rests on someone else's result, carry the grade with it. The TX-03 scheme is the house standard: `[hw]` measured on named hardware, `[sim]` simulation or leakage model only, `[analysis]` analytical or formal result with no implementation, `[norm]` normative standard text, `[adv]` advisory standard text, `[relayed]` the citing author quotes a third party, `[vendor]` unaudited vendor claim. Tabulating a `[sim]` result beside a `[hw]` result without the grades visible overstates demonstrated capability, which is the same defect as fabricating a finding.

Minimum-proof data handling. Capture the least evidence that proves the point: a hash, a record count, a redacted fragment. Never bulk-collect, never retain third-party personal data or secrets beyond what the engagement permits, and redact in reports.

4. Confidence language and the evidence ladder

Incident response is probabilistic. The discipline is to say exactly how confident you are, in words that mean the same thing to every reader.

Standard assessment wording.

Assessment	Meaning	How to write it
CONFIRMED	Direct, reproducible, or provider-validated evidence establishes the event	"Evidence confirms..." - name the artifact and the validation
PROBABLE	Multiple independent indicators support one explanation; alternatives are weaker	"The available evidence makes it probable that..."
POSSIBLE	Some evidence is consistent with the hypothesis; alternatives remain material	"The evidence is consistent with, but does not establish..."
NOT SUPPORTED	Reviewed evidence does not support the hypothesis within stated scope	State what was examined and the limits. Never write "impossible" or "guaranteed clean"

Assessment	Meaning	How to write it
INCONCLUSIVE	Acquisition, logging, time, or evidence quality prevents a defensible conclusion	State the gap, and choose the recovery path on consequence rather than reassurance

The evidence ladder. Each rung tells you something specific and, just as importantly, refuses to tell you other things. Reading a rung for more than it carries is the most common analytical error in intrusion work.

Signal	What it can tell you	What it cannot prove
IP geolocation	What a database believes about a public address	Who controls a machine, or that traffic is malicious
Open connection	A process is communicating with a remote endpoint	That the endpoint is command and control
Process path and signature	Whether a binary looks like expected installed software	That the process has not been injected into or abused
Persistence artifact	How code may survive reboot or logon	That it is malicious, without further context
Execution evidence	What actually ran, and when	Initial access, by itself
Correlated timeline	Multiple independent events supporting one story	Absolute certainty

Do not over-attribute. "This address geolocates to another country" is not equivalent to "an attacker there controls this machine." Geography is enrichment, never proof. A VPN exit, a secure web gateway, a CDN, or a stale geolocation record explains the same observation more simply, and [Section 32](#) gives the procedure for ruling those out first.

Weak signals that need corroboration. Foreign address; port 443; a process named `svchost.exe`; a high-numbered local port; a file under `AppData` or `/tmp`; one potentially-unwanted-application detection; a process using PowerShell or `curl`; battery drain, heat, or a reboot on a mobile device. Each is a lead. None is a conclusion.

Signals that strengthen sharply in combination. Regular beaconing intervals from an application that should not initiate network connections; a new or unsigned binary in a user-writable directory with persistent egress; a persistence mechanism launching that same binary; a process tree where a document reader, browser child, web service, or scripting engine produces an unexpected shell; lookups for newly observed or algorithmically generated domains immediately followed by connections; connections that continue after the owning application is closed; endpoint-security events showing injection, credential access, or control tampering at the same timestamps.

A defensible conclusion answers six questions: which process communicated, what launched it, how it persisted, what privilege it obtained, how it entered, and why the behavior stopped. An operator holding only a country, a process name, or a hunch does not have a conclusion.

5. Operator readiness

Judgment is the instrument. Everything else in this handbook assumes an operator capable of using it, which makes readiness an operational control rather than a wellness topic.

This section is general readiness education. It is not medical advice, mental-health treatment, or a fitness-for-duty standard, and it never justifies denying rest, food, medical care, or accommodation. If someone may hurt themselves or others, use emergency procedures immediately; in the United States, call or text 988 for confidential crisis support, or 911 for immediate physical danger.

Emotion is telemetry, not command authority. Pressure, frustration, fear, and fatigue carry information about risk and unmet needs. They should be read, named, and routed. They should not drive the keyboard.

The pressure sequence. Notice the state early; name it plainly; treat a body response as a signal rather than a verdict; lower arousal before deciding; narrow the mission to the next safe action; choose procedure over mood; and debrief afterward without self-attack - what triggered it, what helped, what changes.

Cognitive load is a security control. Decision quality degrades measurably across a long sequence of choices, and in security operations that degradation shows up as skipped verification steps, accepted shortcuts, and missed anomalies. Self-rate load and act on it:

Level	Operator experience	Action
1 - Clear	Full attention; decisions feel deliberate	Continue
2 - Moderate	Some background noise; decisions still reliable	Continue
3 - Loaded	Working hard; slower, but the double-check instinct is intact	Batch routine decisions
4 - Near capacity	Decisions feel effortful; verification feels like an obstacle	Drop non-safety-critical work; notify the lead
5 - Saturated	Cannot safely make a mission-critical decision	Stop; report to the lead; mandatory recovery break

Risk factors that compound it: three or more consecutive high-stakes decisions without a break; a shift beyond eight hours with no structured recovery; context-switching across four or more active threads; sustained ambiguity with no resolution path; a night of sleep debt carried into the shift; and a post-incident adrenaline crash with no cool-down.

Mitigations that work: batch routine decisions into one block; take a five-to-ten-minute full stop after every ninety minutes of sustained cognitive work; reduce scope rather than speed at level 4; and return to a written procedure when improvisation starts to feel tempting - improvisation under fatigue is a security risk.

Solo and remote operation. Isolation removes the passive check-in a shared workspace provides, and gradual degradation becomes hard to self-detect. Agree a check-in cadence at shift start: scope and cognitive baseline at start, a brief status every ninety minutes, notification within ten minutes of any pressure event, and a three-line shutdown note at the end - what happened, what helped, what needs follow-up. If a check-in is missed, the lead attempts alternate contact within fifteen minutes rather than waiting for the end of shift. Silence is not confirmation of safety. This is a safety control with a transparent, agreed cadence, not surveillance.

Escalate rather than absorb. Strong operators ask for backup before the system fails. Escalate on: inability to safely perform the task; self-rated load at 4 or above for thirty minutes; a security or safety near-miss following a period of elevated stress; severe or persistent symptoms; or increased substance use as a coping mechanism. Getting help is disciplined escalation, not a loss of discipline.

Physical readiness, briefly. Long console sessions and field equipment handling produce the two injury classes operators actually get: repetitive strain and hand or finger injury. Keep wrists neutral and inputs close enough to avoid reaching; change posture every twenty-five to thirty minutes and move for three to five minutes every sixty to ninety; keep hands out of pinch points and the line of fire when racking, lifting, or closing equipment; use the tool rather than fingers; and report numbness, tingling, swelling, or loss of grip strength early rather than working through it. Personal protective equipment is selected from a hazard assessment for the actual task, and damaged or ill-fitting equipment is replaced, not improvised around.

The temporary-field reminder. A hard shift, a long incident, or a deployment is a period of work, not a whole life. There is a next handoff, a next sleep window, and a next door home. Handle the next clean action.

Part II - The Operator Platform

An operator is only as trustworthy as the machine they operate from. Part II builds that machine: the workstation baseline, the field platform, and the workspace and publication discipline that keeps client and incident material from leaking out through the operator.

6. Workstation baseline - Windows

The goal is a workstation that minimizes optional tracking and unnecessary data exposure **without weakening the security floor**. A machine with telemetry reduced but patching, endpoint protection, reputation checking, or certificate validation broken is less private in practice, because compromise becomes more likely.

Before you change anything. Create a restore point or full backup. Record the Windows edition, build number, account type, and encryption status. Confirm the recovery key is stored where the owner can retrieve it and - for organization-owned devices - where the organization can govern it. Never enable encryption without a tested recovery path. On managed devices, identify the owner and the approval path before touching managed settings.

```
# Read-only pre-flight
winver
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, OsBuildNumber
Get-BitLockerVolume
Get-MpComputerStatus | Select-Object AMServiceEnabled, AntivirusEnabled, RealTimeProtectionEnabled
Get-NetFirewallProfile | Select-Object Name, Enabled
```

Three profiles. Choose deliberately; the differences are real.

Profile	Best for	Posture
Balanced	Most users and small business	Optional telemetry, advertising identifiers, activity history, and app permissions reduced. Lowest support risk.
High privacy	Legal, executive, travel, incident-response workstations	Adds strict browser settings, minimal cloud sync, AI capture disabled, consumer surfaces off, stronger account separation. Some convenience features stop working.
Managed fleet	Organization-managed devices	Enforced by policy (Group Policy, MDM, or documented local policy) with update rings, key escrow, and an exception register.

What to reduce. Optional diagnostic data and feedback prompts; advertising identifier and personalised offers; activity history; app-launch tracking and suggested content; location where not required; clipboard history and cross-device sync for anyone handling secrets or client data; consumer surfaces such as search highlights, widgets, spotlight, and store suggestions; and browser personalisation and sync to personal accounts.

AI capture surfaces deserve their own pass. Screenshot-and-analysis features change the exposure model of every window on the machine: hiding a window, using a private tab, or minimizing an application is not a privacy control when screen analysis is enabled. Disable snapshot capture and screen-analysis features, or accept their data-handling model explicitly for a named role. Prohibit pasting credentials, client records, incident evidence, or regulated data into consumer AI surfaces, and write that rule down rather than assuming it.

Application permissions are least privilege. Review camera, microphone, voice activation, notifications, account info, contacts, calendar, messaging, file system, and screen capture. Deny globally where unused; otherwise deny per application. Re-check after installing anything - new applications, browser extensions, conferencing tools,

vendor utilities, VPN clients, password managers, and AI assistants all request access at install time and keep it afterwards.

Treat browser extensions as applications. An extension with page access can read, change, or exfiltrate browser data. Use an allowlist. Remove coupon, shopping, search, screenshot, PDF, and productivity extensions that inspect page content unless a named owner approves them. Use separate browser profiles so personal sync, client work, admin portals, and disposable research do not share cookies, history, or credentials.

What to keep enabled, always. These protect the user, and disabling them to reduce network chatter is a bad trade:

Keep enabled	Why
Operating-system and driver updates	Security patches and reliability fixes through approved channels
Endpoint protection and its intelligence updates	Malware, script, and reputation protection
Firewall profiles (public, private, domain)	Inbound exposure control, especially on laptops that move networks
Reputation-based protection	Warns on malicious sites, downloads, and unwanted applications; it is a security control, not an advertising feature
Disk encryption, Secure Boot, TPM	Protects local data after loss or theft, and protects boot and credential integrity
Certificate validation and root updates	Protects TLS and signed-code trust decisions
Activation, licensing, and required authentication	Keeps the platform and managed applications functioning
Time synchronisation	Security protocols and every log correlation depend on accurate time

Do not run untrusted debloat scripts on client systems. Many disable services with no edition, update, recovery, or rollback testing. If one is used, save the change list, test it in a lab, and document the rollback commands.

Verification and rollback are what make it a baseline rather than a checklist. After applying changes, reboot once, sign in as a standard user, and confirm the machine still works: updates check and install, endpoint protection updates and scans run, reputation protection is on, and the privacy settings reflect the intended state. Capture before-and-after evidence for managed or client devices, and record every exception with an owner, a reason, a compensating control, an expiry date, and a rollback path.

```
# Read-only verification spot checks
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ErrorAction SilentlyContinue
Get-MpComputerStatus | Select-Object AntivirusEnabled, RealTimeProtectionEnabled, IsTamperProtected
Get-NetFirewallProfile | Select-Object Name, Enabled
Get-BitLockerVolume
gpreresult /h "$env:USERPROFILE\Desktop\policy-result.html" # managed devices
```

Privacy drift is the steady state. Feature updates, new applications, browser extensions, setup prompts, and cloud sign-ins all reintroduce settings. Re-review after every feature update and new device, and monthly for extensions, site permissions, linked devices, and sync folders. Vendor and peripheral utilities carry their own telemetry and auto-start helpers; review them separately from the operating system. Keep an inventory of management agents - remote support, endpoint detection, backup, print management, VPN - and explain them separately from consumer tracking, because those records exist to prove security and recover systems.

7. Workstation baseline - Linux

The same principle applies: reduce exposure without breaking the controls that make the host recoverable and observable.

Host baseline.

- Run as a standard user. Elevate for specific administrative actions rather than living as root.
- Full-disk encryption with a tested recovery path, and a strong lock credential.
- Automatic security updates, or a documented patch cadence with evidence of application.
- Minimal installed package set; remove what is not used rather than leaving it listening.
- Host firewall enabled with a default-deny inbound posture and explicit, named exceptions.
- Time synchronisation against a trusted source; log correlation depends on it.
- Audit and journal retention long enough to cover your investigation window (see [Section 27](#)).

Read-only posture check. Run this before changing anything, and keep the output as the before-state.

```
#!/usr/bin/env bash
set -Eeuo pipefail
{
  date --iso-8601=seconds
  hostnamectl
  id
  ip -brief addr
  ss -ltnup                # listening sockets and owning processes
  systemctl --failed
  systemctl list-unit-files --type=service --state=enabled
  systemctl list-timers --all
  awk -F: '$3==0 {print "UID0: "$1}' /etc/passwd
  find / -xdev -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null # SUID/SGID inventory
} > "host-baseline-$(date +%Y%m%d).txt"
```

Least privilege in practice. Grant `sudo` by specific command where the task allows it rather than blanket access; review [sudoers](#) and group membership on a schedule; and keep service accounts non-interactive. Record the SUID and SGID inventory as a baseline - [Section 21](#) turns it into a drift check.

Container and virtualisation awareness. A script that assumes it is on the host will produce misleading results inside a container, and a process view inside a namespace is not the host's process view. Detect the environment explicitly rather than inferring it, and say which view a finding came from.

8. The mobile field platform

A phone running a Linux userland is a genuinely useful operator platform: a portable shell for authorized administration, DNS and certificate checks, scripting, field notes, and supervised lab work. It is not a laptop replacement, and pretending otherwise produces bad field decisions.

The no-root approach is the default. Install a terminal environment from a trusted source, add a userland distribution manager, and install a Linux distribution inside it. Android remains installed; no bootloader unlock, custom firmware, or root is required. Native Linux phone installs are highly device-specific and can permanently damage a device if the wrong image or flashing process is used.

Component	Role	Root required
Android (stock)	Host operating system, untouched	No
Trusted app catalogue	Source for maintained terminal builds	No
Terminal emulator	Linux package environment on Android	No
Userland distro manager	Installs and manages Linux userlands	No
Debian userland	Full package ecosystem for scripting and tooling	No

Install from the official catalogue or project source, never from repackaged download sites. Grant storage access deliberately, update packages at the start of each session, and create a normal user inside the userland rather than working as root - it mirrors production Linux and limits damage from a typo.

What will not work like a laptop. State these limits before someone discovers them mid-task:

- Raw sockets, packet injection, and monitor mode are blocked by the platform's kernel permissions in a no-root setup. This is by design.
- Service supervision behaves differently inside a userland than under a full init system.
- Battery optimisation suspends background work; long jobs need explicit wake handling and still drain the battery and heat the device.
- Large compiles, long captures, and bulk data processing are laptop or lab-VM work.

Practical field setup. A compact Bluetooth keyboard changes the platform from a curiosity to something you will actually use. Keep the tool set minimal: fewer tools means fewer updates, less storage, and less confusion. Start a terminal multiplexer session immediately after login so a screen sleep or focus change does not kill work in progress, and name sessions by task.

Baseline tooling. The useful default set is a text editor, version control, HTTP and DNS clients, a JSON processor, a multiplexer, SSH with key-based authentication, and Python with virtual environments. Prefer Ed25519 keys. Anything that transmits, scans, or tests belongs behind the authorisation check in [Section 2](#), and belongs on the team's approved-tooling list before it is used in the field.

Sensitive material stays off shared storage. Shared Android storage is convenient and more exposed than application-private directories. Keep keys, tokens, credentials, client evidence, and private network details out of it unless encrypted with a clear protection plan. Enable device encryption and a strong lock credential; a lost field phone with plaintext client data is a breach, not an inconvenience.

Rebuild and backup. A field device should be cheap to rebuild. Export the package list, keep notes and configuration in one named directory, archive it with a checksum, and keep the archive out of any location that would republish it. Removing and reinstalling the userland is a normal repair action, not a disaster - as long as the notes were somewhere else.

9. Workspace layout, field notes, and evidence hygiene

A consistent workspace makes evidence defensible and handoffs cheap. Use the same layout on every platform.

```
~/greynoc/
notes/      operator notes, checklists, runbooks
scripts/    helper scripts (bash, python)
logs/       session notes and authorized command output
reports/
  templates/ blank report templates for reuse
  evidence/   raw output and captures; sanitized copies only for sharing
  completed/  finalised deliverables
clients/    per-engagement folders; never plaintext secrets
lab/        home-lab and training artifacts
tools/      custom utilities
```

Add a `README.md` to each engagement folder recording the scope and the authorisation reference. That single file is what makes the work auditable a year later.

The field-note format. Separate observation from conclusion. The most common reporting defect is a note that records an interpretation and loses the underlying fact.

Date / time (with timezone):
 Operator:
 Authorized environment and authorisation reference:
 Purpose:
 Commands or actions performed:
 Observed result:
 Interpretation (explicitly marked as interpretation):
 Follow-up needed:
 Sensitive data captured? yes / no
 Sanitized copy created? yes / no

Evidence handling rules.

Rule	Practice
Work on a copy	Never edit original evidence; work on a duplicate
Hash on capture	Record SHA-256 at capture time so later tampering is detectable
Sanitize before sharing	Remove addresses, hostnames, credentials, customer names, and internal paths
Verify the copy	Review the sanitized copy line by line before it leaves the workspace
No plaintext secrets	Credentials, keys, and tokens never live in plaintext on a field device
Strip metadata	Check document and image metadata before external release
Retain deliberately	Keep only as long as the engagement requires, then dispose per Section 23

Run manifests. Any capture that will support a finding gets a small manifest beside it: run identifier, site, authorisation reference, operator, device, start and stop times, and the hash of each artifact. The manifest is what lets a reviewer reproduce your result without calling you.

10. OPSEC and publication discipline

Operators publish: write-ups, screenshots, conference talks, social posts, and public guides. Publication is where client data most often escapes, and it escapes through details nobody meant to include.

Categories to redact before anything leaves the workspace.

Category	Examples
Network identifiers	Addresses, hostnames, MAC addresses, SSIDs
Credentials and keys	Passwords, API keys, private keys, tokens, hashes
Account identifiers	Username, email addresses, employee identifiers
Client and business data	Customer names, ticket numbers, contract details
Location data	Coordinates, badge identifiers, plates, schedules
Personal identifiers	Faces, reflections in screens, voices in recordings
Filesystem clues	Terminal paths, folder names, internal hostnames in prompts
Document metadata	Author fields, revision history, embedded coordinates
Browser context	Open tabs visible in screenshots, address-bar history

The habit that works: **work on a copy, sanitize the copy, verify the copy, publish only the verified copy.** Use synthetic examples in public demonstrations rather than real targets, and use the documentation address space from [Section 1](#) when an example needs an address.

Timing is part of OPSEC. Delay or avoid posting from live work, travel, incidents, or sensitive sites. A post that is harmless a month later can disclose presence, capability, or schedule today. The same reasoning applies to recording while operating a vehicle or equipment: content creation belongs to a stopped, legally parked, non-operating moment, never to an active driving or operating task. Hands-free is not risk-free, and no post is worth the risk.

Representing the organization. In uniform, in a marked vehicle, or under the GreyNOC name, an operator's conduct is the client's experience of the firm. Communicate findings factually and without exaggeration; stay strictly in scope; escalate anything ambiguous; and if approached by property owners, staff, or law enforcement during authorized field work, be professional and transparent about being on an authorized assessment and contact the engagement lead.

Part III - Defensive Scripting

A defensive script is a small operational product. It has users, failure modes, maintenance needs, and security impact. It should reduce toil, increase visibility, preserve evidence, improve consistency, or help an operator make a safer decision. It should not conceal itself, bypass consent, defeat platform controls, harvest data it does not need, or touch systems outside written permission.

Part III is the craft: how to choose a language, how to build a script you can trust unattended, the patterns most defensive automation reduces to, and a catalog of worked examples mapped to a recognized control structure.

11. Scripting foundations

When a script earns its place. Wherever a task is repetitive, error-prone by hand, or needs to run unattended - collecting system state, parsing logs, checking controls, verifying backups, generating reports. The test is simple: if you can write the task down as a clear, ordered sequence, you can usually script it. If you cannot describe it clearly, define the goal before writing code.

Choosing a language.

Language	Strong fit	Typical operator use
Bash	Linux, Unix, servers, field userlands	File operations, service checks, scheduling glue, small wrappers
PowerShell	Windows and its management ecosystem	Directory, endpoint, and security reporting; object pipelines
Python	Cross-platform, anything with parsing or testing	APIs, log parsing, reusable tooling, data work

Cross-platform task, or one that needs real parsing and tests? Python. Deep inside one operating system? The native shell is simpler and faster. Move from shell to Python when the parsing or the testing becomes awkward - that awkwardness is the signal, and pushing past it is how fragile scripts are born.

Anatomy. Syntax differs; structure does not. Every script has comments, variables, input, output, conditionals, loops, functions, error handling, and an exit code.

Comments explain why, not what.

```
# Weak: restates the obvious
print(name)

# Strong: explains intent
# Display the account currently being processed in the audit report
print(name)
```

Exit codes are the contract with automation. Schedulers, pipelines, and monitoring read the exit code, not your prose. Zero is success; non-zero is failure.

Code	Conventional meaning
0	Success
1	General or catch-all error
2	Misuse - bad arguments or usage
3-125	Application-specific failures you define and document
126	Command found but not executable
127	Command not found
130	Terminated by Ctrl+C (SIGINT)

Three drafts. Every good script grows through the same progression. Skipping straight to clever is how fragile scripts are born.

```
# Draft 1 - make it work
import shutil
total, used, free = shutil.disk_usage("/")
print(used / total * 100)

# Draft 2 - make it readable and decisive
import shutil
THRESHOLD = 80
total, used, _ = shutil.disk_usage("/")
pct = used / total * 100
print(f"WARNING: {pct:.1f}%" if pct >= THRESHOLD else f"OK: {pct:.1f}%")

# Draft 3 - make it safe and automatable
"""disk_check.py -- warn when disk usage exceeds a threshold."""
import shutil, sys

THRESHOLD = 80

def main() -> int:
    total, used, _ = shutil.disk_usage("/")
    pct = used / total * 100
    if pct >= THRESHOLD:
        print(f"WARNING: disk usage {pct:.1f}%")
        return 1
    print(f"OK: disk usage {pct:.1f}%")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

The operator script lifecycle. Scripting is not just writing commands. Plan the outcome, scope, platforms, permissions, and data handled. Prototype the smallest safe command that proves the idea. Harden it with input validation, strict mode, traps, timeouts, and idempotence. Package it with a README, usage, dependencies, checksums, and a version. Deploy it in stages with monitoring, a rollback, and owner sign-off. Observe the result and write down what you learned.

The core procedure, every time.

1. Write the expected result in one sentence.
2. Run the smallest read-only command that proves the starting state.
3. Validate every external input: paths, hostnames, service names, files, API data, and flags.
4. Log time, host, action, result, and error - without exposing secrets.
5. Run in dry-run mode, review the output, then execute with approval where applicable.
6. Verify the final state with an *independent* command and document the result.

Step 6 is the one most often skipped. A successful exit code proves the command ran, not that the business outcome occurred.

Failure modes that recur.

- Assuming interactive shell variables exist under cron, service managers, SSH, or boot sessions.
- Unquoted variables, predictable temporary files, and broad globs.
- Collecting more data than needed, creating privacy and retention problems.
- Treating exit code 0 as proof the outcome succeeded.
- Skipping peer review because the script is short.

12. Analysis before trust - the ten lenses

Run every script through these lenses before you trust it: yours, inherited, or downloaded.

Lens	The question it answers
Purpose	What exactly is this supposed to do?
Environment	What operating system, interpreter, and permissions does it assume?
Dependencies	What modules, tools, keys, or network access does it need?
Input	Where does input come from, and is it trusted and validated?
Logic	Does it make correct decisions, including at the edges?
Output	What does it print, write, or overwrite - and does it leak data?
Security	Does it need privilege, handle secrets, or run remote code?
Performance	Will it scale to the real data volume?
Reliability	Does it handle failure and run safely more than once?
Maintainability	Can the next operator understand and change it?

The injection lens, in detail. The single most important input question: can untrusted input change *which command runs*? Never build a command by gluing strings together.

```
# UNSAFE - invites command injection
import os
target = input("host: ")
os.system("ping " + target)          # "203.0.113.10; rm -rf ~" would execute

# SAFER - arguments as a list; no shell string parsing
import subprocess
target = input("host: ")
subprocess.run(["ping", "-c", "4", target], check=True)
```

Pass arguments as a list and avoid invoking a shell. This one habit eliminates an entire vulnerability class. The same rule holds in shell - quote every expansion - and in PowerShell, where `Invoke-Expression` on untrusted input is the equivalent mistake.

High-risk commands to review carefully in any script.

Command	Why it is dangerous
<code>rm -rf <path></code>	Recursive, forced deletion with no undo - and a variable that expands to empty is catastrophic
<code>Remove-Item -Recurse -Force</code>	The same, on Windows
<code>dd, mkfs, blkdiscard</code>	Can overwrite or wipe entire devices
<code>chmod -R / chown -R</code> on a broad path	Sweeping permission changes break systems in ways that are hard to reverse
Piping a remote script straight to a shell	Runs unseen code instantly; download, read, verify, then run

The safe-execution checklist. Read the whole script and confirm its source. Identify anything that deletes, overwrites, formats, or disables. Run with least privilege, in a lab or with a dry run first. Back up what matters, watch output live, and review logs afterwards.

13. Building to a standard

Start with a header. Name, purpose, author, version, date, requirements, usage. It costs four lines and saves the next operator a phone call.

```
"""
GreyNOC Script
Name: fim.py          Purpose: detect changes to watched files.
Author: GreyNOC       Version: 1.0          Date: 2026-09-20
Requirements: Python 3 Usage: python3 fim.py /etc --init
"""
```

Separate configuration from logic. Put tunables at the top where they are easy to review and change.

```
# Configuration - reviewed separately from logic
AUTH_LOG_PATH      = "/var/log/auth.log"
FAILED_LOGIN_LIMIT = 10
REPORT_FILE        = "failed_login_report.txt"
```

Validate input and log decisions.

```
import logging
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("audit")

value = input("port: ")
if not value.isdigit() or not (1 <= int(value) <= 65535):
    log.warning("Rejected invalid port: %r", value)
else:
    log.info("Accepted port %s", value)
```

Make dangerous actions opt-in. Named flags, not positional mystery. Dry-run by default for anything that writes. Exit non-zero on failure. Structured output for machines and a readable summary for humans. Keep platform-specific logic isolated so the cross-platform path stays clean.

Design for the next operator. The best compliment a script gets is that someone else understood its purpose without calling the author. Boring, predictable, reversible, and documented beats clever.

Verification checklist for any script before it ships.

- Exits non-zero when a required input is missing or invalid.
- Output is understandable by another operator.
- Logs are protected and contain no credentials, tokens, private keys, or unnecessary personal data.
- It can be re-run safely, or it clearly refuses to re-run when that would be unsafe.
- Rollback or stop instructions are documented next to the script.

14. Bash for operators

Strict mode and a cleanup trap open every serious script.

```
#!/usr/bin/env bash
set -Eeuo pipefail          # exit on error, unset var, failed pipe; traps inherit
IFS=$'\n\t'                # safer word-splitting
umask 077                  # new files are owner-only

WORKDIR="$(mktemp -d)"      # private temp directory
cleanup() { rm -rf "$WORKDIR"; }
trap cleanup EXIT          # runs however the script ends
trap 'exit 130' INT        # 128 + SIGINT(2) - EXIT trap then cleans up
trap 'exit 143' TERM       # 128 + SIGTERM(15)
trap 'echo "ERROR line=$LINENO command=$BASH_COMMAND" >&2' ERR

main() {
    echo "starting defensive task in $WORKDIR"
}
main "$@"
```

- `set -e`, `-u`, and `-o pipefail` turn silent failures into loud, early ones. `-E` makes the `ERR` trap fire inside functions and subshells.
- `trap cleanup EXIT` guarantees temporary data is removed however the script ends.
- `IFS=$'\n\t'` prevents space-splitting surprises on filenames and log lines.
- `umask 077` matters whenever the script writes evidence or logs.

Why the signal traps exit rather than clean up. Handling `INT` or `TERM` does *not* terminate a Bash script. Once the handler returns, execution resumes at the next command. The common `trap cleanup EXIT INT TERM` is therefore a trap in both senses: on Ctrl+C the working directory is deleted and the script **keeps running against state that no longer exists**, then exits 0 and reports success for a run the operator cancelled. Put cleanup on `EXIT` only, and let the signal handlers exit - the `EXIT` trap still fires, so cleanup runs exactly once and the exit status honestly reports how the run ended.

```
# Demonstrates the defect: cleanup runs, then the script continues and exits 0
trap cleanup EXIT INT TERM      # WRONG
# Correct: cleanup once, on EXIT; signals terminate with a truthful status
trap cleanup EXIT; trap 'exit 130' INT; trap 'exit 143' TERM
```

Strict mode occasionally needs a local exception - a command whose non-zero exit is expected. Make the exception explicit and narrow rather than disabling strict mode for the whole script:

```
set +e; some_command_that_may_fail; rc=$?; set -e
(( rc == 0 || rc == 1 )) || { echo "unexpected rc=$rc" >&2; exit 1; }
```

Argument parsing with `getopts`. Explicit flags, rejection of unknown flags, and usage on misuse.

```
usage() { echo "usage: $0 -t <target> [-v] [-n]" >&2; exit 2; }
verbose=0; dry_run=0; target=""
while getopts ":t:vn" opt; do
    case "$opt" in
        t) target="$OPTARG" ;;
        v) verbose=1 ;;
        n) dry_run=1 ;;
        *) usage ;;
    esac
done
[[ -n "$target" ]] || usage
```

Input validation. Validate shape before use, and never use substring matching for a security decision - a look-alike value will pass.

```
# Validate a port
[[ "$port" =~ ^[0-9]+$ ]] && (( port >= 1 && port <= 65535 )) \
|| { echo "invalid port: $port" >&2; exit 2; }

# Validate a path stays inside an expected root (resolve first, then compare)
root="/srv/greynoc"
real="$(readlink -f -- "$candidate")" || exit 2
[[ "$real" == "$root"/* ]] || { echo "path outside root: $candidate" >&2; exit 2; }
```

Atomic writes. A half-written file that another process reads is a real failure mode. Write to a temporary file in the same directory, then rename.

```
tmp="$(mktemp "${target}.XXXXXX")"
generate_content > "$tmp"
mv -f -- "$tmp" "$target"      # rename is atomic within a filesystem
```

Lock files and concurrency. Scheduled scripts overlap eventually. Use a lock and fail fast.

```
exec 9>"/var/lock/greynoc-task.lock"
flock -n 9 || { echo "another run is in progress" >&2; exit 0; }
```

Backups before change. Any script that edits a file keeps a timestamped copy first, and says where it went.

```
cp -a -- "$conf" "${conf}.bak.${date +%Y%m%d-%H%M%S}"
```

Quoting rules, condensed. Quote every expansion (`"$var"`, `"${arr[@]}"`). Prefer `[[ ]]` over `[ ]`. Use arrays rather than space-separated strings for argument lists. Use `--` before user-supplied values passed to commands that take options. Prefer `printf` over `echo` for anything that might begin with a dash or contain escapes.

Static review. Run every shell script through ShellCheck before committing. It catches quoting bugs, unsafe patterns, and portability issues the eye misses, and it belongs in the pre-commit hook and in CI (Section 22).

15. Python for operators

The production-shaped skeleton.

```
#!/usr/bin/env python3
"""Production-shaped CLI skeleton."""
import argparse, logging, sys
from pathlib import Path

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger("tool")

def parse_args() -> argparse.Namespace:
```

```

p = argparse.ArgumentParser(description="What this tool does.")
p.add_argument("path", type=Path)
p.add_argument("--threshold", type=int, default=80)
p.add_argument("--dry-run", action="store_true")
return p.parse_args()

def main() -> int:
    args = parse_args()
    log.info("Starting on %s (dry-run=%s)", args.path, args.dry_run)
    # ... work ...
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

- `argparse` over hand-parsing `sys.argv`: free help text, types, and validation.
- `logging` over `print`: levels, timestamps, and routing to files or a SIEM.
- `pathlib.Path` for cross-platform paths; type hints for clarity and tooling.
- A `--dry-run` flag so the tool can show intent without acting.
- Pin dependencies and work inside a virtual environment.

Subprocess safely. Argument lists, no shell, explicit timeouts, and checked results.

```

import subprocess

def run(args: list[str], timeout: int = 30) -> subprocess.CompletedProcess:
    """Run a command with no shell, a timeout, and captured output."""
    return subprocess.run(args, capture_output=True, text=True,
                          timeout=timeout, check=False)

proc = run(["systemctl", "is-active", "--quiet", service])
active = proc.returncode == 0

```

Resilient external calls. Retry transient failures with exponential backoff, and re-raise on the final attempt rather than swallowing the error.

```

import time

def with_retry(fn, attempts: int = 3, base: float = 0.5):
    """Retry fn with exponential backoff; re-raise on final failure."""
    for i in range(attempts):
        try:
            return fn()
        except Exception:
            if i == attempts - 1:
                raise
            time.sleep(base * (2 ** i))

```

Path handling. Resolve before you compare, and refuse anything that escapes the expected root.

```

from pathlib import Path

def safe_child(root: Path, candidate: str) -> Path:
    p = (root / candidate).resolve()
    if not p.is_relative_to(root.resolve()):
        raise ValueError(f"path escapes root: {candidate}")
    return p

```

Events as JSON Lines. One JSON object per line is the friendliest format for both humans and ingestion pipelines: streamable, greppable, and trivially parsed.

```

import json, sys

def emit(event: dict) -> None:

```

```
json.dump(event, sys.stdout, separators=(",", ":"))
sys.stdout.write("\n")
```

Tests earn their keep on logic. Unit-test the decision functions - thresholds, parsers, classifiers - rather than the plumbing. A parser with a test suite survives a log-format change; one without it fails silently at 3 a.m.

Packaging and deployment. Create a virtual environment per tool, pin dependencies in a requirements file, and record the exact interpreter version in the README. For scheduled runs, call the interpreter inside the virtual environment by absolute path rather than depending on an activated shell.

16. PowerShell for operators

```
<#
.SYNOPSIS Restart a service safely, with -WhatIf support.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
    [Parameter(Mandatory)][ValidateNotNullOrEmpty()]
    [string]$ServiceName
)
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

try {
    if ($PSCmdlet.ShouldProcess($ServiceName, "Restart service")) {
        Restart-Service -Name $ServiceName
        Write-Output "Restarted $ServiceName"
    }
}
catch {
    Write-Error "Failed: $_"
    exit 1
}
```

- `SupportsShouldProcess` gives `-WhatIf` and `-Confirm` for free - the native dry-run.
- `Set-StrictMode -Version Latest` plus `$ErrorActionPreference = "Stop"` surface silent failures.
- Validate parameters with attributes such as `[ValidateNotNullOrEmpty()]` and `[ValidateRange()]`.
- Use approved verbs and comment-based help so discovery works.
- Avoid `Invoke-Expression` on anything untrusted.

Objects, not text. PowerShell's advantage is the object pipeline. Emit objects and let the caller format them; do not pre-format into strings that the next stage has to re-parse.

```
# Emits objects a caller can filter, sort, or export
Get-CimInstance Win32_Service |
    Select-Object Name, State, StartMode, StartName, PathName
```

Static review. Run scripts through PSScriptAnalyzer in the pre-commit hook and in CI, the same way Bash goes through ShellCheck.

A note on execution policy. Execution policy is a safety rail against accidental execution, not a security boundary. Scope any change narrowly and document it; never instruct a user to disable it machine-wide as a convenience.

17. Patterns and models

Most defensive scripts worth trusting are instances of a few reusable models. Recognizing them turns a blank page into an assembly job.

Model	Idea	Where it shows up
Baseline and diff	Snapshot a known-good state; alert on deviation	File integrity, configuration drift, port and account auditing, wireless inventory
Observer / monitor	Watch a source; emit events on conditions	Log triage, health checks, authentication-failure detection
Pipeline / filter	Pass data through composable stages	Log parsing, enrichment, reporting
Idempotent converge	Describe desired state; re-running is safe	Provisioning, hardening, cleanup
Retry with backoff	Tolerate transient failure gracefully	API and network calls
Fail-safe / fail-secure	Choose the safe default when uncertain	Access decisions, error paths
Least privilege	Grant only what the task needs	Every script that touches a real system
Defense in depth	Layer independent safeguards	Validation plus sandboxing plus monitoring

Baseline and diff is the workhorse. It is the single most important defensive model in this handbook and it recurs everywhere: file integrity in [Section 21](#), configuration drift, account and port auditing, wireless inventory in [Section 34](#), and detection coverage in [Section 35](#). Snapshot a curated, owner-confirmed known-good state; store it where the monitored system cannot quietly rewrite it; recompute; and report added, removed, and changed distinctly. Three rules make it trustworthy:

1. **Curate the baseline.** A baseline taken from an unverified system encodes whatever was already wrong. Confirm ownership and expected configuration before marking anything authorized.
2. **Store it out of reach.** A baseline that lives on the host it monitors can be edited by whoever compromised that host.
3. **Key on the stable identifier.** Names are chosen and duplicated; hardware identifiers, inventory identifiers, and hashes are not. Keying on the wrong field produces confident nonsense.

Fail-safe versus fail-secure is a deliberate choice. A fail-safe lock opens on power loss (safety first); a fail-secure lock stays locked (security first). Decide which your script is when an error interrupts it mid-run, and say so in the header. The default behavior on error is a security decision whether or not you made it consciously.

Idempotence in practice. The test is not "can I run it twice" but "does running it twice leave the same state and cause no additional harm." Check-then-act, guard destructive steps behind a state test, and make the second run a no-op that says so.

```
# Idempotent: converge to desired state rather than blindly applying
if ! grep -qxF "$desired_line" "$conf"; then
  cp -a -- "$conf" "${conf}.bak.${date +%Y%m%d-%H%M%S}"
  printf '%s\n' "$desired_line" >> "$conf"
  echo "[CHANGED] appended directive"
else
  echo "[OK] already present; no change"
fi
```

18. Secrets management

Credentials are the highest-value thing a script touches. Keep them out of source, out of arguments, and out of logs.

Approach	Rating	Notes
Managed secret store or vault	Best	Central rotation, access control, audit trail
Managed or workload identity	Best	No long-lived secret on disk at all

Approach	Rating	Notes
Operating-system keyring	Good	Per-user encrypted storage
Environment variable	Acceptable	Convenient; beware process listings and child inheritance
Restricted config file (<code>chmod 600</code>)	Acceptable	Outside the repository; never world-readable
Hardcoded in source	Never	Compromised the moment it is committed

Never pass secrets as command-line arguments. They are visible to other users in the process list and frequently land in shell history. Prefer an environment variable, a file descriptor, or standard input.

```
import os
api_key = os.getenv("GN_API_KEY")      # from the environment, never the source
if not api_key:
    raise SystemExit("Set GN_API_KEY before running.")
```

A committed secret is compromised, permanently. It remains in history even after deletion, and history gets cloned, mirrored, and backed up. The order of operations is fixed: **rotate first, scrub second**. Removing the line does not un-leak a credential that already reached a shared repository.

Keep them out of history and memory longer than needed. Prevention beats cleanup: keep secrets out of `argv` and out of log statements so they never reach history or logs in the first place. Where a shell is configured to ignore space-prefixed commands, that helps for a one-off, but it is a convenience, not a control. If a secret did land in history, remove the entry *and* rotate the credential - the entry may already be in a backup.

```
token = os.environ.pop("GN_API_TOKEN", None) # remove from env after read
try:
    ...
finally:
    del token                                # drop the reference
```

19. Logging, telemetry, and SIEM-friendly output

Logs are how unattended scripts earn trust. Make them structured, timestamped, and free of secrets, so they feed straight into a pipeline.

```
import json, logging, sys

class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({
            "ts": self.formatTime(r), "level": r.levelname,
            "logger": r.name, "msg": r.getMessage(),
        })

h = logging.StreamHandler(sys.stdout)
h.setFormatter(JsonFormatter())
log = logging.getLogger("gn"); log.addHandler(h); log.setLevel(logging.INFO)
log.info("scan complete")      # -> {"ts": "...", "level": "INFO", ...}
```

What every operational log line carries. Time in a single, stated timezone - prefer UTC in ISO 8601 for anything that will be correlated across systems; host; the action; the result; and the error when there is one. What it never carries: secrets, tokens, full request bodies, session material, or personal data beyond what the task requires. A log file is just another place a credential can leak.

Normalize to one schema. Multi-source logs are only queryable once they share a shape. Define a small target schema and parse each source into it; drop or hash fields that carry secrets; emit newline-delimited JSON so downstream tools can stream it; and send malformed lines to a quarantine file for review rather than letting them vanish. A silently dropped line is an ingestion gap that will be mistaken for an absence of activity.

```
#!/usr/bin/env python3
"""Normalize a log source into the common operator schema."""
import json, sys, datetime

SCHEMA = ("ts", "host", "source", "severity", "actor", "action", "detail")

def normalize(line: str, source: str) -> dict:
    parts = line.rstrip().split(" ", 5)
    if len(parts) < 2:
        raise ValueError("too short")
    ts = datetime.datetime.fromisoformat(parts[0]).isoformat()
    rec = dict(zip(SCHEMA, [ts, parts[1], source] + parts[2:]))
    return {k: rec.get(k, "") for k in SCHEMA}

for line in sys.stdin:
    try:
        print(json.dumps(normalize(line, sys.argv[1])))
    except Exception as e:
        sys.stderr.write(f"quarantine: {e}: {line}")
```

Two audiences, two outputs. Machines want structured events; humans want a readable summary. Emit both - structured to stdout or a log file, summary to stderr or a final block - rather than compromising on a format that serves neither.

Rotation. Any script that writes a log on a schedule needs a rotation plan before it is deployed, or it becomes the disk-space incident. Use the platform's rotation facility rather than reimplementing it, and set retention to match the investigation window from [Section 27](#), not to "forever."

20. Scheduling and service integration

A script that works interactively and fails under a scheduler is the single most common operator surprise. The cause is almost always environment: scheduled contexts have a minimal `PATH`, no interactive shell configuration, a different working directory, and no terminal.

Rules that prevent it.

- Use absolute paths for interpreters, binaries, and data.
- Set the variables you depend on inside the script rather than inheriting them.
- Do not assume the working directory; `cd` explicitly or use absolute paths throughout.
- Capture both stdout and stderr to a log; a scheduled failure with no output is undiagnosable.
- Test by invoking the script the way the scheduler will, not from your shell.

```
# cron - absolute interpreter, absolute script, both streams captured
0 2 * * * /usr/bin/python3 /opt/greynoc/verified_backup.py >> /var/log/gn_backup.log 2>&1
```

Service managers give you more than cron. A timer plus a service unit provides logging, dependency ordering, resource limits, failure handling, and a clean status query. Prefer it for anything that matters.

```
# /etc/systemd/system/gn-portdrift.service
[Unit]
Description=GreyNOC listening-port drift check
After=network-online.target

[Service]
```

```
Type=oneshot
ExecStart=/opt/greynoc/port_drift.sh /etc/greynoc/allowed_ports.txt
User=greynoc
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
```

```
# /etc/systemd/system/gn-portdrift.timer
[Unit]
Description=Run the port drift check hourly

[Timer]
OnCalendar=hourly
Persistent=true

[Install]
WantedBy=timers.target
```

`NoNewPrivileges`, `ProtectSystem`, `ProtectHome`, and `PrivateTmp` are least privilege applied to a scheduled job: they cost one line each and shrink the blast radius of a defect in your own script.

Mobile and field scheduling. On a field device, background execution is governed by the platform's power management, not by your scheduler. Long-running work needs an explicit wake mechanism, a visible indication that it is running, and an obvious way to stop it. Keepalive behavior must never be used to hide activity, bypass ownership, or defeat policy - a field script that is not visible and not easy to stop is a policy problem regardless of intent. Prefer longer intervals and event-driven checks over tight loops, and skip non-critical work when the battery is low.

21. The defensive script catalog

Every script below is **read-and-alert by design**: it observes, baselines, and reports rather than taking destructive action. That makes it safe to deploy early and run on a schedule. Adapt paths, thresholds, and alerting to your environment, and test in a lab first.

The catalog maps to the NIST Cybersecurity Framework 2.0 functions (Govern, Identify, Protect, Detect, Respond, Recover) so automation ties back to a recognized control structure.

Script	CSF function	What it does
Failed-login monitor	Detect	Counts authentication failures per source; flags brute force
File integrity monitor	Detect	Hashes critical files; reports change, addition, removal
TLS expiry checker	Protect	Warns before certificates expire and cause outages
Listening-port baseline	Detect	Differs open ports against an approved allowlist
Verified backup	Recover	Creates backups and proves integrity with checksums
Resource health monitor	Detect	Tracks disk, memory, and CPU against thresholds
Indicator enrichment	Respond	Matches indicators against local intelligence with a pluggable feed hook

Script	CSF function	What it does
Secret scanner	Protect	Finds keys and tokens accidentally committed to a repository
Configuration drift detector	Protect	Compares live configuration to an approved baseline
New-account auditor	Detect	Flags newly created or newly privileged accounts
SUID/SGID auditor	Identify	Inventories privilege-granting binaries against an allowlist
Scheduled-job auditor	Detect	Baselines scheduled jobs; reports unexpected changes
Web availability probe	Detect	Checks reachability, status, latency, and certificate health
Log triage summarizer	Respond	Buckets log levels and flags error spikes
Alert triage scorer	Respond	Ranks alerts by explainable weighted factors
Evidence manifest builder	Respond	Hashes captured artifacts into a verifiable manifest

21.1 Failed-login monitor (Bash).

```
#!/usr/bin/env bash
# failed_login_monitor.sh : report sources over a failed-login threshold.
set -Eeuo pipefail
AUTH_LOG="${1:-/var/log/auth.log}"
THRESHOLD="${2:-10}"
[[ -r "$AUTH_LOG" ]] || { echo "ERROR: cannot read $AUTH_LOG" >&2; exit 2; }

# grep exits 1 on "no matches", which is the normal quiet-night result, not an error.
# Collect first so strict mode does not abort the run before it reports.
counts="$(grep -a "Failed password" "$AUTH_LOG" \
    | grep -aoE "from [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" \
    | awk '{print $2}' | sort | uniq -c | sort -rn || true)"

alerts=0
while read -r count ip; do
    [[ -z "${count:-}" ]] && continue
    if (( count >= THRESHOLD )); then
        echo "[ALERT] $ip -- $count failed attempts"
        alerts=$(( alerts + 1 ))
    fi
done <<< "$counts"

echo "[INFO] Scan complete (${alerts} over threshold)"
exit $(( alerts > 0 ? 1 : 0 ))    # 0 = clean scan, 1 = alerting, 2 = could not scan
```

Exit codes are the contract, so get the quiet case right. The obvious one-pipeline version of this script is wrong in a way that only shows up in production: under `pipefail`, a log with no matching failures makes `grep` return 1, and a run where every source is below the threshold leaves the `while` loop carrying the false arithmetic test's status. Both are *normal*, *healthy* results, and both abort the script under `set -e` before it prints anything - so a scheduler records the monitor as failed every quiet night and healthy only while an attack is in progress. Collect the counts first, tolerate the empty case explicitly, and return a status that distinguishes "clean scan" from "alerting" from "broken."

For automated blocking, pair this with a maintained, purpose-built tool rather than scripting bans by hand - the edge cases (allowlists, lockout windows, shared egress addresses) are easy to get wrong, and a self-inflicted lockout is a genuine outage. The detection side of this pattern is [D&R-02](#); distributed, low-and-slow variants are [D&R-01](#).

21.2 File integrity monitor (Python).

```
#!/usr/bin/env python3
"""fim.py : baseline and detect changes to watched files."""
import argparse, hashlib, json, sys
from pathlib import Path

def sha256(p: Path) -> str:
    h = hashlib.sha256()
    with p.open("rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()

def snapshot(paths):
    return {str(p): sha256(p)
            for base in paths for p in Path(base).rglob("*") if p.is_file()}

def main() -> int:
    ap = argparse.ArgumentParser(description="File integrity monitor.")
    ap.add_argument("paths", nargs="+")
    ap.add_argument("--baseline", default="fim_baseline.json")
    ap.add_argument("--init", action="store_true")
    args = ap.parse_args()
    cur, bf = snapshot(args.paths), Path(args.baseline)
    if args.init or not bf.exists():
        bf.write_text(json.dumps(cur, indent=2))
        print(f"[INFO] Baseline written: {bf} ({len(cur)} files)")
        return 0
    old = json.loads(bf.read_text())
    for p in sorted(cur.keys() - old.keys()):
        print(f"[ALERT] ADDED {p}")
    for p in sorted(old.keys() - cur.keys()):
        print(f"[ALERT] REMOVED {p}")
    for p in sorted(k for k in cur.keys() & old.keys() if cur[k] != old[k]):
        print(f"[ALERT] CHANGED {p}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

Store the baseline off-host or read-only. A baseline on the monitored system can be rewritten by whoever compromised it, and the check then reports "no change" forever. Use a reviewed path list rather than a broad recursive sweep, or real drift is lost in noise.

21.3 TLS certificate expiry checker (Python).

```
#!/usr/bin/env python3
"""cert_expiry.py : warn before a TLS certificate expires."""
import argparse, socket, ssl, sys
from datetime import datetime, timezone

def days_left(host: str, port: int = 443) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=8) as s:
        with ctx.wrap_socket(s, server_hostname=host) as ss:
            cert = ss.getpeercert()
            exp = datetime.strptime(cert["notAfter"], "%b %d %H:%M:%S %Y %Z")
            return (exp.replace(tzinfo=timezone.utc) - datetime.now(timezone.utc)).days

def main() -> int:
    ap = argparse.ArgumentParser(description="TLS expiry checker.")
    ap.add_argument("host")
    ap.add_argument("--warn", type=int, default=21)
    a = ap.parse_args()
    try:
```

```

        d = days_left(a.host)
    except Exception as e:
        print(f"[ERROR] {a.host}: {e}")
        return 2
    if d < 0:
        print(f"[ALERT] {a.host}: certificate EXPIRED"); return 1
    if d <= a.warn:
        print(f"[WARN] {a.host}: expires in {d} days"); return 1
    print(f"[OK] {a.host}: {d} days remaining")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

The same handshake position is where post-quantum readiness becomes observable. Extending this probe to record the negotiated group is the cheapest crypto-inventory telemetry an operator can add; see [Section 38](#) and [PB-01](#).

21.4 Listening-port baseline and drift (Bash).

```

#!/usr/bin/env bash
# port_drift.sh : flag listening ports not on the allowlist.
set -Eeuo pipefail
ALLOWLIST="${1:-allowed_ports.txt}" # one port per line
[[ -r "$ALLOWLIST" ]] || { echo "ERROR: missing $ALLOWLIST" >&2; exit 1; }

ss -ltn | awk 'NR>1 {n=split($4,a,"."); print a[n]}' | sort -un \
| while read -r port; do
    [[ -z "$port" ]] && continue
    grep -qx "$port" "$ALLOWLIST" || echo "[ALERT] Unexpected port: $port"
done
echo "[INFO] Port audit complete"

```

A listening port is not the same as a reachable port. Confirm reachability from another authorized host before concluding exposure, and map the port to its owning process before concluding anything at all ([Section 25](#)).

21.5 Verified backup (Bash).

```

#!/usr/bin/env bash
# verified_backup.sh : archive, checksum, and verify.
set -Eeuo pipefail
SRC="${1:?usage: verified_backup.sh <source-dir> <dest-dir>}"
DEST="${2:?usage: verified_backup.sh <source-dir> <dest-dir>}"
stamp="$(date +%Y-%m-%d_%H%M%S)"; archive="$DEST/backup_${stamp}.tar.gz"
mkdir -p "$DEST"
tar -czf "$archive" -C "$(dirname "$SRC")" "$(basename "$SRC")"
sha256sum "$archive" > "$archive.sha256"
sha256sum -c "$archive.sha256"
echo "[INFO] Verified: $archive ($(du -h "$archive" | cut -f1))"

```

A backup you have never restored is a hope, not a backup. The checksum proves integrity, not recoverability - schedule a periodic restore into a scratch location and record the result. This is the control that decides whether [D&R-18](#) ends in recovery or in a rebuild.

21.6 Resource health monitor (Python).

```

#!/usr/bin/env python3
"""health_monitor.py : disk and memory check with exit codes."""
import shutil, sys
from pathlib import Path

DISK_WARN, MEM_WARN = 85, 90

def disk_pct(path: str = "/") -> float:
    t, u, _ = shutil.disk_usage(path)
    return u / t * 100

```

```
def mem_pct() -> float:
    info = {}
    for line in Path("/proc/meminfo").read_text().splitlines():
        k, v, *_ = line.replace(":", "").split()
        info[k] = int(v)
    return (info["MemTotal"] - info["MemAvailable"]) / info["MemTotal"] * 100

def main() -> int:
    rc = 0
    d = disk_pct(); print(f"[{'WARN' if d >= DISK_WARN else 'OK'}] Disk {d:.1f}%")
    rc |= d >= DISK_WARN
    try:
        m = mem_pct(); print(f"[{'WARN' if m >= MEM_WARN else 'OK'}] Mem {m:.1f}%")
        rc |= m >= MEM_WARN
    except FileNotFoundError:
        print("[INFO] Memory check skipped (no /proc/meminfo)")
    return int(rc)

if __name__ == "__main__":
    sys.exit(main())
```

21.7 Indicator enrichment (Python). Matches indicators against a local intelligence list and exposes a pluggable hook for whatever feed your program licenses. No paid service is assumed.

```
#!/usr/bin/env python3
"""ioc_enrich.py : match indicators against local intelligence."""
import argparse, json, sys
from pathlib import Path

def load_intel(path: Path) -> dict:
    # JSON: {"203.0.113.10": "known-scanner", "<sha256>": "sample-label"}
    return json.loads(path.read_text()) if path.exists() else {}

def feed_lookup(indicator: str) -> str | None:
    # Plug a licensed feed here; return a label or None.
    return None

def main() -> int:
    ap = argparse.ArgumentParser(description="Indicator enrichment.")
    ap.add_argument("indicators", nargs="+")
    ap.add_argument("--intel", type=Path, default=Path("intel.json"))
    a = ap.parse_args()
    intel, hits = load_intel(a.intel), 0
    for ioc in a.indicators:
        label = intel.get(ioc) or feed_lookup(ioc)
        if label:
            hits += 1
        print(json.dumps({"indicator": ioc, "match": label}))
    return 1 if hits else 0

if __name__ == "__main__":
    sys.exit(main())
```

21.8 Secret scanner (Python). A defensive scan you run against your own repositories to catch leaks before they ship.

```
#!/usr/bin/env python3
"""secret_scan.py : find committed secrets in a tree."""
import argparse, re, sys
from pathlib import Path

PATTERNS = {
    "Cloud access key": re.compile(r"AKIA[0-9A-Z]{16}"),
    "Private key": re.compile(r"-----BEGIN (?:(RSA |EC |OPENSSH )?PRIVATE KEY-----"),
    "Generic API key": re.compile(r"(?i)(api[_-]?key|secret)\s*[:=]\s*['\"]?[0-9a-zA-Z]{16,}"),
    "Bearer token": re.compile(r"(?i)bearer\s+[0-9a-zA-Z._-]{20,}"),
}
SKIP = {".git", "node_modules", "__pycache__", "venv"}

def main() -> int:
    ap = argparse.ArgumentParser(description="Secret scanner.")
```

```

ap.add_argument("root", type=Path)
a = ap.parse_args()
found = 0
for p in a.root.rglob("*"):
    if not p.is_file() or any(s in p.parts for s in SKIP):
        continue
    try:
        text = p.read_text(errors="ignore")
    except Exception:
        continue
    for line_no, line in enumerate(text.splitlines(), 1):
        for name, rx in PATTERNS.items():
            if rx.search(line):
                found += 1
                print(f"[ALERT] {name}: {p}:{line_no}")
print(f"[INFO] Scan complete ({found} potential secrets)")
return 1 if found else 0

if __name__ == "__main__":
    sys.exit(main())

```

A hit means **rotate first, scrub second** (Section 18). Removing the line does not un-leak a credential that already reached a shared repository or its history.

21.9 Configuration drift detector (Python).

```

#!/usr/bin/env python3
"""config_drift.py : compare configs to an approved baseline."""
import argparse, hashlib, json, sys
from pathlib import Path

def digest(p: Path) -> str:
    return hashlib.sha256(p.read_bytes()).hexdigest()

def main() -> int:
    ap = argparse.ArgumentParser(description="Config drift detector.")
    ap.add_argument("files", nargs="+")
    ap.add_argument("--baseline", default="config_baseline.json")
    ap.add_argument("--init", action="store_true")
    a = ap.parse_args()
    cur = {f: digest(Path(f)) for f in a.files if Path(f).is_file()}
    bf = Path(a.baseline)
    if a.init or not bf.exists():
        bf.write_text(json.dumps(cur, indent=2))
        print(f"[INFO] Baseline written ({len(cur)} files)")
        return 0

    old = json.loads(bf.read_text()); drift = 0
    for f in sorted(cur.keys() - old.keys()):          # appeared since baseline
        drift += 1
        print(f"[ALERT] ADDED {f}")
    for f in sorted(old.keys() - cur.keys()):          # baselined file now missing
        drift += 1
        print(f"[ALERT] REMOVED {f}")
    for f in sorted(cur.keys() & old.keys()):          # present in both, content changed
        if cur[f] != old[f]:
            drift += 1
            print(f"[ALERT] DRIFT {f}")
    print("[OK] No drift" if not drift else f"[INFO] {drift} change(s)")
    return 1 if drift else 0

if __name__ == "__main__":
    sys.exit(main())

```

Report removal as loudly as modification. Iterating only over the files that currently exist is the easy mistake here: a baselined configuration that has been *deleted* never appears in `cur`, so the comparison skips it and prints `[OK] No drift` for exactly the destructive change the detector exists to catch. Compare in all three directions - added, removed, changed - the way Section 21.2 does.

21.10 New-account and privilege auditor (Bash).

```
#!/usr/bin/env bash
# account_audit.sh : baseline users and UID-0 accounts; report drift.
set -Eeuo pipefail
BASE="${1:-account_baseline.txt}"
cur="$(getent passwd | awk -F: '{print $1":"$3}' | sort)"
if [[ ! -f "$BASE" ]]; then
    echo "$cur" > "$BASE"; echo "[INFO] Baseline written"; exit 0
fi
echo "[INFO] UID 0 accounts:"; awk -F: '$3==0 {print "  "$1}' /etc/passwd
diff <(echo "$cur") "$BASE" | grep '^<' | sed 's/^< /[ALERT] NEW USER: /' \
|| echo "[OK] No new accounts"
```

21.11 SUID/SGID auditor (Bash).

```
#!/usr/bin/env bash
# suid_audit.sh : inventory SUID/SGID binaries against an allowlist.
set -Eeuo pipefail
ALLOW="${1:-suid_allowlist.txt}"
touch "$ALLOW"
find / -xdev -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | sort \
| while read -r bin; do
    grep -qx "$bin" "$ALLOW" || echo "[ALERT] Unexpected SUID/SGID: $bin"
done
echo "[INFO] SUID/SGID audit complete"
```

21.12 Scheduled-job auditor (Bash). Scheduled tasks are a first-class persistence surface (D&R-15), which makes baselining them high-value for a few lines of shell.

```
#!/usr/bin/env bash
# cron_audit.sh : baseline scheduled-job sources and report changes.
set -Eeuo pipefail
BASE="${1:-cron_baseline.txt}"
collect() {
    sha256sum /etc/crontab 2>/dev/null
    # Hash the contents, not just the paths: a job file can be rewritten in place.
    find /etc/cron.d /etc/cron.daily /etc/cron.hourly /etc/cron.weekly /etc/cron.monthly \
        -type f -print0 2>/dev/null | xargs -0 -r sha256sum 2>/dev/null
    for u in $(cut -f1 -d: /etc/passwd); do
        crontab -l -u "$u" 2>/dev/null | sed "s/^/[$u] /"
    done
    systemctl list-timers --all --no-pager 2>/dev/null
}
cur="$(collect | sort)"
if [[ ! -f "$BASE" ]]; then
    echo "$cur" > "$BASE"; echo "[INFO] Baseline written"; exit 0
fi
diff <(echo "$cur") "$BASE" | grep '^<' | sed 's/^< /[ALERT] NEW OR CHANGED JOB: /' \
|| echo "[OK] No scheduled-job changes"
```

Baseline the contents, not the filenames. Listing the job directories with `find` alone records only paths, so rewriting an existing job to call a different binary leaves the snapshot byte-for-byte identical and the auditor reports no change - while the persistence mechanism has in fact been repointed. Hashing each job file makes an in-place edit visible, which is the whole point of the check.

21.13 Web availability and certificate probe (Python).

```
#!/usr/bin/env python3
"""web_probe.py : reachability, status, latency, certificate days."""
import sys, time, ssl, socket
from urllib.request import urlopen
from urllib.parse import urlparse
from datetime import datetime, timezone

def cert_days(host: str, port: int = 443) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((host, port), timeout=8) as s:
        with ctx.wrap_socket(s, server_hostname=host) as ss:
```

```

        exp = datetime.strptime(ss.getpeercert()["notAfter"],
                                "%b %d %H:%M:%S %Y %Z")
    return (exp.replace(tzinfo=timezone.utc) - datetime.now(timezone.utc)).days

def probe(url: str) -> int:
    t0 = time.time()
    try:
        with urlopen(url, timeout=10) as r:
            ms = (time.time() - t0) * 1000
            parts = urlparse(url)
            # Check the certificate on the port the URL actually uses, not always 443.
            days = (cert_days(parts.hostname, parts.port or 443)
                    if parts.scheme == "https" else "n/a")
            print(f"[OK] {url} {r.status} {ms:.0f}ms cert={days}d")
            return 0
    except Exception as e:
        print(f"[ALERT] {url} unreachable: {e}")
        return 1

def main() -> int:
    return max((probe(u) for u in sys.argv[1:]), default=0)

if __name__ == "__main__":
    sys.exit(main())

```

21.14 Log triage summarizer (Python).

```

#!/usr/bin/env python3
"""log_triage.py : bucket log levels and flag error spikes."""
import argparse, re, sys
from collections import Counter
from pathlib import Path

LEVEL = re.compile(r"\b(DEBUG|INFO|WARN|WARNING|ERROR|CRITICAL)\b")

def main() -> int:
    ap = argparse.ArgumentParser(description="Log triage.")
    ap.add_argument("logfile", type=Path)
    ap.add_argument("--error-threshold", type=int, default=50)
    a = ap.parse_args()
    counts = Counter()
    for line in a.logfile.read_text(errors="ignore").splitlines():
        m = LEVEL.search(line)
        if m:
            counts[m.group(1).replace("WARNING", "WARN")] += 1
    for lvl in ("CRITICAL", "ERROR", "WARN", "INFO", "DEBUG"):
        if counts[lvl]:
            print(f" {lvl:<8} {counts[lvl]}")
    errs = counts["ERROR"] + counts["CRITICAL"]
    if errs >= a.error_threshold:
        print(f"[ALERT] Error spike: {errs} (>= {a.error_threshold})")
        return 1
    print("[OK] No error spike")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

21.15 Alert triage scorer (Python). Ranks a queue by a small, explainable score so attention goes to the highest signal first. The score must be bounded, reproducible, and accompanied by its factor breakdown - a hidden weighting nobody can explain during an incident is worse than no scoring at all.

```

#!/usr/bin/env python3
"""triage_score.py : rank alerts by explainable weighted factors."""
import json, sys

W = {"asset": 3, "confidence": 2, "blast": 2, "novelty": 1}

def score(a: dict):
    s = sum(W[k] * a.get(k, 0) for k in W)
    return min(s, 10), {k: a.get(k, 0) for k in W}

```

```
alerts = [json.loads(l) for l in sys.stdin if l.strip()]
for a in alerts:
    a["score"], a["why"] = score(a)
for a in sorted(alerts, key=lambda x: x["score"], reverse=True):
    print(json.dumps(a))
```

Version-control the weights, log the scoring decisions, and tune from real outcomes rather than intuition. See [Section 28](#) for how the score fits the triage workflow.

21.16 Evidence manifest builder (Bash). Turns captured artifacts into something a reviewer can verify without calling you.

```
#!/usr/bin/env bash
# evidence_manifest.sh : hash artifacts into a verifiable manifest.
set -Eeuo pipefail
umask 077
STORE="${EVIDENCE_DIR:?evidence directory required}"
mkdir -p "$STORE"
: > "$STORE/manifest.tsv"
find "$STORE" -type f ! -name 'manifest.tsv' -print0 \
| while IFS= read -r -d '' f; do
    printf '%s\t%s\t%s\n' \
        "$(date -Is)" "$(sha256sum "$f" | cut -d ' ' -f1)" "$(basename "$f")" \
        >> "$STORE/manifest.tsv"
done
echo "[INFO] manifest written: $STORE/manifest.tsv"
```

22. Supply chain, build, and release

Operator tooling is software, and it ships to production systems. Treat it that way.

Dependency and CI hygiene.

- Pin dependencies to exact versions and verify checksums; review what you install.
- Lint and test in CI - ShellCheck, a Python linter, PSScriptAnalyzer, plus unit tests on logic.
- Sign or hash released scripts so consumers can verify integrity before running.
- Least-privilege CI: scoped tokens, no long-lived secrets in the pipeline.

Pre-commit checks catch what review misses. A hook that runs the linters, the secret scanner from [Section 21.8](#), and the fast tests turns most defects into a five-second local failure instead of a review comment.

Versioning and changelog. Use semantic versioning: patch for fixes, minor for backwards-compatible additions, major for breaking changes. A changelog entry names what changed, why, and what an operator must do differently. Changelog discipline is a release control, not documentation housekeeping - it is how a responder in six months knows whether the behavior they are seeing was intended.

Release the artifact, not the working copy. Build into an immutable, versioned directory, checksum it, and switch a symlink. Rollback is then switching the symlink back, which is fast and obvious under pressure.

```
#!/usr/bin/env bash
# release.sh : build into an immutable versioned directory and switch the symlink.
set -Eeuo pipefail
VERSION="${1:?usage: release.sh <version>}"
ROOT=/opt/greynoc
REL="$ROOT/releases/$VERSION"

[[ -e "$REL" ]] && { echo "release $VERSION already exists" >&2; exit 1; }
mkdir -p "$REL"
cp -a build/. "$REL/"
# Exclude the manifest from its own digest, or it hashes itself while still empty.
( cd "$REL" && find . -type f ! -name SHA256SUMS -print0 | sort -z \
```

```
| xargs -0 sha256sum > SHA256SUMS )
( cd "$REL" && sha256sum -c SHA256SUMS >/dev/null )
ln -sfh "$REL" "$ROOT/current"
echo "[INFO] released $VERSION -> $ROOT/current"
```

The manifest must not be inside its own manifest. The redirection creates `SHA256SUMS` before `find` walks the tree, so an unfiltered `find . -type f` includes the manifest and records the digest of its empty self. Writing the remaining lines immediately invalidates that entry, the verification step fails, and under strict mode every release aborts before the symlink switch - a release pipeline that can only ever fail. Excluding it by name fixes the ordering; `sort -z` also makes the manifest deterministic so two builds of identical content produce identical manifests.

Deployment rings. Stage the rollout: a canary host, then a small representative group, then the fleet. Define the promotion criterion before you start - what must be true to move to the next ring - and the abort criterion that sends you back. A rollout with no stated abort criterion is not a staged rollout.

Rollback is a plan, not an intention. Write it down next to the script: the exact command that reverts, who can authorise it, what evidence to capture first, and how you will know the revert worked. Test it at least once in a lab. A rollback nobody has exercised is a hypothesis.

The release quality gate. Before a version ships: static checks pass, tests pass, the secret scan is clean, the changelog entry exists, the version is bumped, artifacts are checksummed, the rollback is documented, and an owner has signed off. Anything that cannot meet the bar gets flagged to the owner as an explicit exception with a reason - not silently shipped around.

23. Operational privacy and cleanup

Scripts create artifacts: temporary files, logs, output, caches, and shell history. In authorized work those artifacts often contain client data or credentials. Cleaning up is part of the job - both to protect that data and to leave authorized systems as you found them.

Cleanup here means lawful, documented hygiene: protecting collected data, removing test artifacts you created on authorized systems with permission, and keeping your own workstation clean. It is never about evading detection or defeating an incident response. Authorized engagements are documented and coordinated, not hidden.

The artifact lifecycle: create -> secure -> use -> sanitize -> verify. Designing with the lifecycle in mind is what makes cleanup reliable instead of an afterthought.

Artifact	Where it hides	Cleanup approach
Temporary files	<code>/tmp</code> , working directories, caches	<code>mktemp</code> plus trap-based removal; RAM-backed storage for sensitive scratch
Output and reports	Result files, exports	Store in an encrypted container; sanitize on disposal
Logs	Application logs, system logs, SIEM	Redact before writing; control retention
Shell history	Shell history files	Keep secrets out of <code>argv</code> ; scrub and rotate if one slipped in
Environment	Exported variables, child processes	Unset after use; never pass secrets in <code>argv</code>
Credentials in memory	Process memory, swap	Minimize lifetime; full-disk and swap encryption

Secure handling while running. Create sensitive temporary data with a restrictive umask and owner-only permissions; prefer a private `mktemp -d` directory. For highly sensitive scratch data, work in RAM-backed storage so nothing durable reaches the disk. Set short timeouts and minimize how long credentials live in memory.

Cleanup that runs whether the script finishes, errors, or is interrupted.

```
#!/usr/bin/env bash
# Self-cleaning template
set -Eeuo pipefail
umask 077
WORK="$(mktemp -d)"
cleanup() {
    rm -rf "$WORK" 2>/dev/null || true
    unset GN_API_TOKEN 2>/dev/null || true
}
trap cleanup EXIT          # cleanup runs exactly once, however the script ends
trap 'exit 130' INT        # signals terminate; the EXIT trap still cleans up
trap 'exit 143' TERM

echo "[INFO] Working in $WORK"
# --- sensitive work confined to $WORK ---
echo "[INFO] Done; cleanup runs automatically on exit"
```

Cleanup is bound to `EXIT` alone for the reason given in [Section 14](#): handling `INT` or `TERM` does not end the script, so a combined `EXIT INT TERM` handler wipes the working directory and then lets the script carry on against deleted state.

```
#!/usr/bin/env python3
"""Self-cleaning template."""
import os, sys, tempfile

def main() -> int:
    with tempfile.TemporaryDirectory() as work: # removed on exit
        os.chmod(work, 0o700)
        token = os.environ.pop("GN_API_TOKEN", None)
        try:
            print(f"[INFO] Working in {work}")
            return 0
        finally:
            del token # drop the reference

if __name__ == "__main__":
    sys.exit(main())
```

Secure deletion, current guidance. Deleting a file usually just unlinks it; the data remains until overwritten. The classic advice to "overwrite with random data" no longer holds on modern storage, and getting this wrong gives false confidence. On solid-state and flash media, wear-levelling and over-provisioning mean an overwrite may never touch the original cells. On journaling and copy-on-write filesystems, and anywhere snapshots or backups exist, single-file overwrite tools are unreliable.

NIST SP 800-88 Rev. 2 (effective September 2025), which defers technique selection to IEEE 2883-2022, takes a risk-based view and favours **cryptographic erase**: keep sensitive data only inside an encrypted volume and destroy the key.

Goal	Reliable approach
Protect engagement data at rest	Full-disk or volume encryption
Destroy a set of sensitive files	Keep them in an encrypted container; destroy the container and key
Sanitize a whole solid-state device	The drive's own secure-erase or sanitize command, then verify
Sanitize a magnetic disk	Overwrite is acceptable on non-journalled magnetic media; verify

Goal	Reliable approach
Highest assurance, end of life	Physical destruction per NIST SP 800-88 Rev. 2

The practical operator habit: never let sensitive client data exist in plaintext at rest. Store it inside an encrypted volume from the start, and cryptographic erase becomes as simple as destroying the key.

Post-engagement hygiene checklist.

- Confirm all test artifacts placed on authorized systems are removed, and documented as removed.
- Move collected data into an encrypted container; remove plaintext working copies.
- Rotate any credentials generated, used, or exposed during the engagement.
- Verify cleanup against a checklist and capture it in the engagement record.
- Sanitize or encrypt the workstation storage that held client data, per your data-handling policy.

Document what you did and what you removed. In authorized work, a clean, auditable trail of your own actions protects both the client and you.

Part IV - Cyber Defense Operations

Part IV is the operational core: knowing what you have, reducing what is reachable, defending the identities that reach it, building detections that fire, triaging what they produce, and running the incident when one of them is real.

24. Asset and attack-surface inventory

You cannot defend, detect on, or recover what you have not inventoried. Every other section in Part IV assumes this one exists.

What an operator inventory records. For each asset: a stable identifier, an owner, its role, its exposure (internal, external, third-party managed), its data sensitivity, the security controls that apply to it, and its lifecycle state. The owner field is the one most often skipped and the one that decides whether a finding gets fixed.

Stable identifiers matter more than names. Names are chosen, duplicated, and changed. Key the inventory on identifiers the asset cannot casually alter: hardware identifiers, serial numbers, certificate fingerprints, cloud resource identifiers, package or image digests. Keying on a friendly name produces confident nonsense the first time two things share one.

Inventories that pay for themselves.

Inventory	What it enables	Built by
Host and endpoint	Patch coverage, agent coverage, ownership	Management tooling plus Section 7 posture check
Listening services	Exposure reduction (Section 25)	Section 21.4 port baseline
Accounts and privilege	Identity defense (Section 26), D&R-06, D&R-14	Section 21.10 account audit
Scheduled jobs and services	Persistence detection, D&R-15	Section 21.12 job audit
Certificates	Outage prevention, crypto inventory (Section 38)	Section 21.3 , Section 21.13
Software and dependencies	Vulnerability and supply-chain response	Build pipeline, Section 22
Wireless estate	Rogue detection, perimeter leakage (Section 34)	Curated baseline, Section 34

Inventory	What it enables	Built by
Cryptographic inventory	Post-quantum readiness (Section 38)	PB-01
AI assets and agents	AI governance and risk tiering	PB-21

Shadow assets are the normal case, not the exception. Unsanctioned access points, personal cloud accounts, developer test services, and unapproved AI tools appear because the sanctioned path was slower or unclear. Treat a discovered workaround as a signal about the approved path: capture it, assess the risk, and convert the safe and useful patterns into approved workflows. Punishing the discovery just moves it further out of sight.

Two operator archetypes shape what you will find. Process-led users protect continuity and keep institutional memory; their risk is local copies, legacy workarounds, and personal procedure that drifts from the system of record. Discovery-led users adopt quickly and expose friction fast; their risk is skipped documentation, unapproved tools, and changes that are hard to audit. Neither is better, and neither maps to age. Design for both: pair a fast-start path with a deeper reference path, explain why controls exist, and make the safe path faster than the workaround - otherwise capable users will build the workaround for you.

25. Network exposure reduction

A service is exposed when something is listening **and** a network path allows traffic to reach it. Both halves matter, and confusing them is the most common error in this work.

Discovery from two directions. Local commands tell you what the machine believes is listening. A check from another authorized host tells you what can actually be reached. You need both.

```
# Windows: listening sockets, owning process, owning service
Get-NetTCPConnection -State Listen |
  Sort-Object LocalPort |
  Format-Table LocalAddress, LocalPort, OwningProcess
Get-Process -Id <PID>
Get-CimInstance Win32_Service | Where-Object ProcessId -eq <PID> |
  Select-Object Name, DisplayName, State, StartMode
```

```
# Linux: listening sockets with owning process
ss -ltnup
```

Map every port to a process and a purpose before acting. A port number does not identify an application. Decide from the finding, not the number:

Finding	Preferred action
Service is not needed	Stop and disable the service, then remove unnecessary allow rules
Needed by administrators only	Restrict by VPN, management subnet, jump host, or source address
Needed internally, not broadly	Limit scope to the required hosts or segments
Unknown	Investigate process owner, dependency, and recent change history before blocking

Removing the service beats blocking the port. A disabled service cannot be re-exposed later by a firewall rule change, a new network path, or a VPN route. Firewall blocking is a useful second layer, not the primary control.

Legacy discovery and name-resolution protocols. Legacy broadcast name-resolution services are frequently unnecessary in environments that use modern directory-based resolution, and they leak host and service information. Review and disable where no legacy dependency exists - but verify first. Some older applications,

printers, file-sharing workflows, backup agents, and mixed-version networks still depend on them. Related-but-distinct protocols deserve their own decisions and their own testing rather than being swept up in one change. And file-sharing services are a separate review from name resolution; blocking them blindly breaks domain services and administrative workflows.

Development, test, and AI-tooling ports are a live operator concern. Coding assistants, test runners, local model servers, notebooks, vector stores, and prototype frameworks start temporary services so software can be previewed or inspected. These are meant for local use. Risk appears when they bind to all interfaces, run without authentication, expose debug features, or outlive the test.

The workflow that keeps this safe:

1. List listening ports **before** starting the tool. That is your baseline.
2. Start the assistant, dev server, notebook, or local model tool.
3. Map every new port to a process owner and a purpose.
4. Confirm whether each binds to loopback or to all interfaces.
5. From another authorized host, test whether it is actually reachable.
6. Stop what is unused; restrict what must remain.
7. Clean up: close sessions, stop containers, remove temporary firewall rules, and record any exception with an expiry.

```
# Bind local test services to loopback by default
python3 -m http.server 8000 --bind 127.0.0.1
```

Browser remote-debugging endpoints, unauthenticated notebook servers, model-serving APIs, vector databases, and training dashboards are the highest-value of these to get wrong: each can expose source, data, model artifacts, or an execution path. [PB-20](#) covers the multi-tenant and inference-infrastructure version of this problem in depth.

Common mistakes that cause outages.

- Blocking remote management without console or out-of-band access, locking administrators out.
- Assuming a local listening socket is externally reachable - verify from another host.
- Assuming a port number identifies the application.
- Leaving demo applications, notebooks, tunnels, containers, or debug ports running after a test.
- Making a permanent allow rule for a temporary troubleshooting session.

Change discipline. Record the before-state, identify required services, test on one device or a small pilot group, schedule production changes in a window, keep an out-of-band recovery path, and document who approved the change and what success looks like. Time-box exceptions: every open management, file-sharing, or test port gets an owner, a reason, a source scope, a review date, and a rollback plan.

26. Identity and access defense

Identity is where most intrusions start and where most of them are finished. The controls are unglamorous and they work.

The identity baseline.

Control	Operator standard
Authentication strength	Phishing-resistant factors for privileged and high-risk users; hardware security keys are the strongest practical option, passkeys an acceptable alternative

Control	Operator standard
SMS and voice codes	Not a primary factor for privileged access - vulnerable to phishing, signalling attacks, and number takeover
Privilege separation	No daily-driver administrative account; separate credential for administrative work
Least privilege	Role-scoped access, reviewed on a schedule, with joiners-movers-leavers actually executed
Session management	Bounded session lifetimes; the ability to revoke sessions and tokens centrally and fast
Recovery paths	Inventoried and protected - recovery email, phone, contacts, and backup factors are attack surface
Service and non-human identities	Inventoried, scoped, rotated, and owned like any other account
Account PIN and port-out lock	Set with the carrier for high-risk users; number takeover bypasses SMS factors entirely

Backup factors are a custody problem. Maintain at least two hardware factors where the platform requires a backup, and store them separately under controlled custody. Losing every trusted device and key can cause permanent lockout, so plan custody and recovery *before* enrolment, not after.

Detection is where identity defense becomes observable. The library carries the logic: D&R-01 password spraying, D&R-02 brute force, D&R-04 credential stuffing, D&R-05 impossible travel, D&R-06 privilege escalation, D&R-12 post-compromise mailbox and OAuth abuse, and D&R-14 directory credential theft. PB-05 covers signature and token integrity, including the algorithm-confusion class that turns a token check into a no-op.

Non-human identity deserves explicit attention. Service accounts, API tokens, CI credentials, and autonomous agents now outnumber human accounts in most estates, and they rarely have an owner, a rotation schedule, or a revocation path. Inventory them (Section 24), scope them to the narrowest permission that works, rotate them on a schedule you actually run, and make sure someone can answer "who owns this token" without a search party. Agent and connector authorisation is its own defect class - PB-18 covers the confused-deputy and connector-scope cases that recur in agentic systems.

Identity containment is a separate workstream from host containment. Locking a local password does not invalidate SSH keys, active sessions, API tokens, or application credentials. Section 32 covers the sequence.

27. Detection engineering

A detection is a hypothesis about adversary behavior, expressed so a machine can evaluate it and an analyst can act on it. The playbook library carries the hypotheses; this section is the practice around them.

Behavior over signature. Signatures match what an adversary used last time. Behavioral detections match what the technique requires, and survive the adversary changing tooling. Every D&R-NN playbook in the library is built this way, which is why they translate across platforms.

The lifecycle.

1. **Hypothesis.** Name the behavior and the technique. Cite the technique with its name, ID, and collection version per CONVENTIONS Section 4 - never bare.
2. **Telemetry check.** Confirm the data exists before writing logic. This is the step that kills most detections, and it is better to kill one here than to deploy a rule that can never fire.
3. **Logic.** Write it against the normalized schema (Section 19), not against one source's quirks.

4. **Validation on history.** Run it against historical data. A rule that has never been evaluated against real data has an unknown false-positive rate, which means it has an unknown cost.
5. **Tuning.** Establish the baseline, set thresholds from *your* data, and document exclusions with an owner and a reason.
6. **Deployment.** Version-control it, stage it, and record what it is supposed to catch.
7. **Feedback.** Revisit after every confirmed true positive and every false positive.

Telemetry sufficiency, concretely. Before trusting a detection, answer: Does the log source exist and is it enabled? Is the specific event class being recorded, or filtered out by configuration? Does retention cover the investigation window? Is the rule deployed, enabled, and not suppressed? Would this rule have fired on the last known instance of this behavior?

The absence of a Sysmon-style event is not proof an action did not occur - an event type may be disabled or heavily filtered. Audit rules vary widely; determine which are active before drawing conclusions from missing events. This caution applies with unusual force to physical-layer media, where TX-01 and TX-02 make the point that an empty queue for an uninstrumented band or span is a coverage gap, not a clean result.

Thresholds are local. Published absolute values are starting points, not settings. Every threshold in a playbook is a deviation from a locally recorded baseline. Record your baseline, set the threshold from it, and re-derive after significant environment change.

MITRE mapping discipline. Technique names, IDs, and tactic assignments all change between versions, so none of the three is cited bare. State the version you verified against, in the artifact, next to the mapping. Two specifics from the current baseline that routinely bite:

- ATT&CK v19 retired the "Defense Evasion" tactic. TA0005 became **Stealth**, and TA0112 **Defense Impairment** was split out. Neither substitution is automatic.
- v19 renumbered techniques, not just tactics. T1562 **Impair Defenses** no longer exists; its tool-tampering sub-technique was promoted to top-level T1685 **Disable or Modify Tools**, and T1070.001 was reparented to T1685.005. A redirect from a retired ID to its replacement is the authoritative remapping.

Never sweep a rename across a table. Verify each technique individually - v19 also *removed* a tactic from some techniques rather than renaming it. ATLAS carries the same discipline plus a **maturity** field; quote it when severity is in question, because a technique marked *Feasible* has not been observed in the wild and a report implying otherwise is fabricating impact.

False positives are a design input, not a nuisance. Every playbook in the library carries a false-positive section because the FP profile determines whether a detection is deployable. Document the benign causes, tune against them, and record every exclusion with an owner, a reason, and a review date. An unowned, undated exclusion becomes a permanent blind spot.

Detection as code. Rules live in version control with tests, review, and a deployment pipeline - the same standards as Section 22. That is what makes the purple-team replay loop in Section 35 possible.

28. Alert triage and scoring

Triage is the decision about where finite attention goes. Done badly it produces alert fatigue, which is a control failure with a friendly name.

Score by explainable factors. The Section 21.15 scorer ranks on asset criticality, signal confidence, blast radius, and novelty. The score must be bounded, reproducible from its inputs, and carry its factor breakdown so the triager sees *why*. Hidden weights nobody can explain during an incident are worse than no scoring.

The triage sequence.

1. **Corroborate.** Confirm the observation across independent signals. Rule out a one-off artifact before spending an hour on it.
2. **Classify.** Known-good, known-bad, or unknown. Use the inventory from [Section 24](#); most "unknown" is an inventory gap rather than an adversary.
3. **Prioritize.** By consequence and confidence together. A low-confidence signal against a crown-jewel asset can outrank a high-confidence signal against a lab host.
4. **Document.** Identifier, time, source, observation, interpretation marked as interpretation, and the action taken.
5. **Decide.** Close with a reason, tune with an owner, or escalate to [Section 29](#).

Escalate on consequence, not on certainty. A weak technical signal still warrants immediate protective action when the consequence is large - privileged access, regulated data, safety systems, or an imminent operational deadline. [Section 33](#) applies this explicitly to executive mobile devices, where the decision rule is that you do not need proof before taking temporary protective action.

Alert fatigue is measurable. Track the proportion of alerts closed as false positive, time to first action, and the number of rules that have never produced a true positive. A rule with no true positives in a meaningful window is either mistuned, miswritten, or watching a behavior that does not occur in your estate - all three are findings, and all three are fixable.

Multi-stage correlation. Single alerts describe events; incidents are sequences. [D&R-10](#) covers kill-chain correlation across entities, which is the difference between "a user failed authentication" and "a spray succeeded, a token was minted, and a mailbox rule was created."

29. Incident response - the first hour

The first job is not to solve the incident. It is to stop the situation getting worse while preserving enough evidence to explain it later.

The core rule: do not destroy your evidence. If a host is actively under attack and the stakes are high, prefer network isolation over immediately powering it off. Memory may contain injected code, decrypted configuration, active sockets, and credentials that disappear at shutdown. Do not randomly delete files, clear logs, uninstall software, or run cleaner utilities before you know what happened. If you must contain quickly: isolate first, eradicate second.

The first five minutes.

1. **Establish a timestamp and an operator log.** Record local time and timezone, hostname, logged-in user, and what caused the suspicion. Record commands as you run them. A plain text file is sufficient when no case system exists.

```
``powershell Get-Date; hostname; whoami ``
```

```
``bash date --iso-8601=seconds; hostnamectl; id ``
```

1. **Decide whether immediate isolation is justified.** Isolate on strong evidence of ongoing unauthorized control: destructive activity, confirmed credential theft, a remote shell you did not establish, an unknown process beaconing to a known-malicious destination, active data staging, or an adversary changing accounts and security controls.

If forensic evidence matters, disconnect the network or move the host to a containment segment rather than shutting it down. If human safety or destructive malware is involved, stopping the system may take priority over evidence preservation - record who decided, when, and why.

1. Capture volatile network state.

```
``powershell Get-NetTCPConnection | Sort-Object State, OwningProcess Get-NetUDPEndpoint | Sort-Object OwningProcess arp -a; ipconfig /all; route print ``
```

```
``bash ss -tupna; ip addr show; ip route show; ip neigh show ``
```

1. **Capture process state.** Parent-child relationships are often more revealing than a process name. An office application spawning a shell, a web server spawning a command interpreter, or a service daemon launching a downloader followed by a newly written binary each deserve immediate attention.

```
``powershell Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, Name, ExecutablePath, CommandLine | Sort-Object ProcessId ``
```

```
``bash ps -eo pid,ppid,user,lstart,cmd --sort=pid ``
```

1. **Check the obvious security controls.** Do not treat the absence of an alert as proof the machine is clean. The value is correlation: did a detection occur at the same time as the suspicious process or connection?

The first fifteen minutes. Move from "what is happening now" to "what mechanism is making it happen." Build a shortlist of suspicious processes, accounts, services, scheduled jobs, scripts, and remote endpoints. For each candidate, answer enough to either explain it or escalate it:

Question	Why it matters
What process owns the connection?	Code executes the communication; the process is more useful than the address
What is the executable path?	Malicious code often runs from user-writable or deceptive locations
Who signed the binary?	Useful context, though signed software can still be abused
What is the parent process?	Remote execution and living-off-the-land abuse produce unusual ancestry
How did it persist?	Persistence explains why it returns after a reboot or a process kill
What account context?	User, system, service, and container contexts change the blast radius
What changed recently?	File, service, job, package, account, and log changes anchor the timeline

Command and control is a behavior, not a country. It may use dedicated infrastructure, a cloud platform, a CDN, a VPN, a compromised server, a messaging API, DNS, HTTPS, WebSockets, SSH, or software the organization already permits. The registered country of an address is weak attribution evidence (Section 4).

A useful hunt asks four questions in order: which process communicated; what started that process; what mechanism keeps it available to the adversary; and what else happened at the same time. That order prevents tunnel vision around address reputation. `D&R-08` carries the beaconing detection logic.

Containment that preserves evidence. Prefer centralized isolation where it exists. On a standalone host, physical or network isolation is often faster and safer than composing dozens of firewall rules while an adversary is active. Disable the persistence mechanism before moving the payload, and never block an address merely because it is unfamiliar - shared cloud and CDN addresses affect legitimate software and are weak indicators without process-level evidence.

30. Host investigation - Windows

Windows provides enough built-in visibility for a strong first pass without installing third-party tooling. Map connections to process identifiers, then map those to paths and command lines.

Network to process.

```
Get-NetTCPConnection -State Established |
  Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort, OwningProcess |
  Sort-Object OwningProcess

$ids = 1234, 5678
Get-CimInstance Win32_Process |
  Where-Object { $_.ProcessId -in $ids } |
  Select-Object ProcessId, ParentProcessId, Name, ExecutablePath, CommandLine |
  Format-List
```

A connection becomes interesting when ownership does not fit the application. A browser reaching a CDN is ordinary. A newly created executable in a temporary directory reaching the same external address every thirty seconds is not. For a generic service host, the service behind that instance tells you more than the process name.

Binary validation.

```
Get-AuthenticodeSignature 'C:\Path\program.exe' |
  Format-List Status, StatusMessage, SignerCertificate, Path
Get-FileHash 'C:\Path\program.exe' -Algorithm SHA256
Get-Item 'C:\Path\program.exe' |
  Select-Object FullName, Length, CreationTime, LastWriteTime, VersionInfo | Format-List
```

A valid signature means the file carries a cryptographically valid signature from the listed publisher. It does not prove the software is harmless: legitimate signed tools are abused, signed binaries can be vulnerable, and a signed process can host injected code. Record the hash before quarantine or removal so the same file can be correlated across endpoints without moving the executable around.

Process-tree patterns worth investigating.

Parent	Child	Why
Office application	Script interpreter or command shell	Possible malicious document or macro execution
Web server worker	Command shell or script interpreter	Possible web-application remote execution or web shell (D&R-16)
Browser	Unexpected binary from a download or temporary path	Possible drive-by or malicious download execution
Service host	Unknown binary in a user-writable path	Unusual service persistence
Scheduler	Script engine with encoded or hidden arguments	Possible scheduled persistence (D&R-15)

Persistence enumeration. If a process returns after you terminate it or reboot, find the persistence before killing it again. Enumerate startup entries, run keys, scheduled tasks with their actions, services with their binary paths and accounts, and management-instrumentation event subscriptions. Permanent event subscriptions are uncommon on most workstations; they are not automatically malicious, but unknown command-line or script consumers deserve careful investigation.

```
Get-CimInstance Win32_StartupCommand | Select-Object Name, Command, Location, User
Get-ScheduledTask | ForEach-Object {
  foreach ($a in $_.Actions) {
```

```
[PSCustomObject]@{ Task = "$($_.TaskPath)$($_.TaskName)"
    Execute = $a.Execute; Args = $a.Arguments }
}
}
Get-CimInstance Win32_Service |
    Select-Object Name, DisplayName, State, StartMode, StartName, PathName
Get-CimInstance -Namespace root/subsubscription -ClassName __FilterToConsumerBinding
```

Event log review. Process creation (where command-line auditing is enabled), script-block and operational script logs, service installation, and logon events, correlated on time. Remote logons are not inherently malicious - investigate source address, logon type, account, workstation name, and whether the activity fits expected administration.

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688;
    StartTime=(Get-Date).AddHours(-24)} | Select-Object TimeCreated, Message
Get-WinEvent -FilterHashtable @{LogName='System'; Id=7045;
    StartTime=(Get-Date).AddDays(-7)} | Select-Object TimeCreated, Message
```

Recent executable and script writes in user, program-data, and temporary paths, bounded by a time window. Recursive searches are expensive - narrow the path and the window on production systems.

Where pre-existing telemetry exists, use it. A system-monitoring agent configured *before* the incident provides process creation, network connection, file creation, image load, and DNS correlation that nothing else gives you. Do not install one mid-incident on a high-stakes forensic target without considering how the installation changes the evidence. And remember configuration matters: an event type may be disabled or filtered, so absence is not proof.

Containment and eradication, in order. Disable the suspicious scheduled task; stop and disable the confirmed malicious service; block a confirmed malicious destination. Then, only after the mechanism is identified with reasonable confidence: remove persistence, quarantine the payload, reset affected credentials, patch the entry point, and verify the activity does not return.

Move a payload to an evidence holding directory rather than deleting it, and hash it on the way. A manually created holding directory is not quarantine and does not prevent execution - it is only appropriate after execution has been stopped and persistence disabled. In higher-assurance environments, use the endpoint platform's quarantine or acquire a forensic image instead of handling the file manually.

31. Host investigation - Linux

The same evidence ladder applies; the command names differ and the logic is identical. Socket to process, process to executable and parent, process to persistence, then timeline.

Sockets and owning processes.

```
sudo ss -tupna
sudo lsof -nP -i
```

Inspect the owning process.

```
ps -fp 1234
tr '\0' ' ' < /proc/1234/cmdline; echo
readlink -f /proc/1234/exe
ls -l /proc/1234/cwd
pstree -aps 1234
```

File and package validation.

```
sha256sum /path/to/suspect
file /path/to/suspect
stat /path/to/suspect
dpkg -S /usr/bin/example || rpm -qf /usr/bin/example
dpkg -V package-name || rpm -V package-name
```

Package verification can reveal files that differ from the vendor package, but configuration files legitimately change. Treat verification output as a lead, not a verdict.

Persistence enumeration. Service units and timers; scheduled jobs at system and user scope; shell profiles and autostart entries; and authorized SSH keys.

```
systemctl list-unit-files --type=service
systemctl list-timers --all
systemctl cat suspicious.service

crontab -l; sudo crontab -l
sudo ls -la /etc/cron.d /etc/cron.daily /etc/cron.hourly

grep -RniE 'curl|wget|bash -c|python|perl|nc|socat|/tmp|/dev/shm' \
  ~/.bashrc ~/.profile ~/.bash_profile /etc/profile /etc/profile.d 2>/dev/null

find /home /root -path '*/.ssh/authorized_keys' -type f -print 2>/dev/null
```

Do not delete unfamiliar keys blindly. Servers carry automation, backup, deployment, and break-glass keys. Record the key, its fingerprint, the file owner, permissions, and timestamps; compare against authorized administration records; then remove only the confirmed unauthorized line.

Logs and authentication.

```
sudo journalctl --since '24 hours ago'
sudo journalctl -p warning..alert --since '24 hours ago'
sudo grep -iE 'Accepted|Failed|Invalid user|session opened|sudo' /var/log/auth.log | tail -n 300
last -ai | head -n 100
sudo journalctl _COMM=sudo --since '24 hours ago'
```

Remote execution investigation. On servers, remote code execution usually becomes visible as a service process spawning something it should not: a web server spawning a shell, an application runtime spawning a downloader, a database service writing an executable into a temporary directory. Find the causal chain from remote request or login to executed code.

```
ps -eo pid,ppid,user,lstart,cmd --forest
sudo find /tmp /var/tmp /dev/shm -xdev -type f -mtime -2 \
  -printf '%TY-%Tm-%Td %TH:%TM:%TS %u %m %s %p\n' 2>/dev/null | sort -r
sudo tail -n 500 /var/log/nginx/access.log # paths vary by distribution
```

Correlate request timestamps with process start times and file creation times. A suspicious request is much stronger evidence when it is immediately followed by a service child process and a newly written payload.

Hidden activity, carefully. Do not assume every discrepancy means a rootkit. Start with simpler explanations - deleted-but-running files, mount namespaces, containers, package updates, permissions - and escalate only when multiple independent views disagree.

```
sudo lsof +L1 # deleted-but-running executables
lsmod
sed -n '1,200p' /proc/1234/maps
```

Containment. Stop and disable the confirmed malicious unit; block a confirmed malicious destination; lock a compromised local account. Never improvise firewall policy on a remote production system without an out-of-band recovery path - you can lock yourself out as effectively as you lock out an adversary. And locking a local password does not invalidate SSH keys, active sessions, API tokens, or application credentials; identity containment is a separate workstream (Section 32).

Audit telemetry. Linux audit records can connect execution, account actions, privilege use, and configuration changes to users and processes - when configured before the incident. Determine which rules are active before drawing conclusions from missing events.

```
sudo auditctl -s
sudo ausearch -ts today
sudo aureport -au
```

32. Timeline, credential response, and the recovery decision

Build the timeline. Timelines turn a pile of artifacts into an incident story. Use one timezone - prefer UTC for multi-system incidents - and record each event with source, timestamp, host, user or process, action, and confidence.

Time (UTC)	Source	Observed event	Interpretation
14:02:11	Web access log	Unusual POST to a vulnerable endpoint	Possible initial trigger
14:02:12	Process log	Web worker spawns a shell	Strong remote-execution evidence
14:02:13	Filesystem	New binary written to a temporary path	Payload staging
14:02:15	Scheduled job	Persistence created	Adversary durability
14:02:18	Network	New process connects externally	Possible command and control
14:07:40	Security log	New administrative account	Privilege and persistence expansion

The sequence matters. *Remote request -> process start -> file write -> persistence -> outbound connection -> credential use* is a very different story from a benign updater that creates a process and contacts its vendor's distribution network.

Rule out the environment before escalating a geographic anomaly. Network conditions imitate compromise. A VPN changes public address and geolocation; a secure web gateway makes many hosts appear to originate from one address; DNS filtering redirects domains; browser extensions and corporate proxies alter what sites see. Document the egress path first.

```
netsh winhttp show proxy
Get-DnsClientServerAddress
```

```
env | grep -i proxy
resolvectl status 2>/dev/null || cat /etc/resolv.conf
```

The worked case: a user is told a site is unavailable in a particular country, and reconnecting the VPN changes the result. Record the public egress address before changing anything; capture socket-to-process mappings; record VPN exit, DNS, and proxy state; reconnect and record the new address; and compare whether the behavior

changes at the same time as the egress address. If the symptom follows the address, prioritize the network explanation. Only treat it as command and control when independent host evidence supports unauthorized code or control.

Credential and identity response. If the adversary reached administrator, system, root, a directory account, a cloud token, a browser session, or an SSH key, assume credential material was exposed. Removing malware without rotating compromised secrets leaves a clean way back in.

1. Identify which accounts were logged in or used while the compromise was active.
2. Revoke active sessions and tokens through the identity provider - not just password resets.
3. Rotate passwords, API keys, service-account secrets, and SSH keys according to impact.
4. Review newly added authentication factors, recovery options, forwarding rules, delegate access, OAuth grants, and application passwords. [D&R-12](#) covers this surface in depth.
5. Check neighbouring systems for reuse of the same credentials or keys.

Scope without probing. Once privileged credentials are implicated, the incident may extend beyond the first host. Do not begin aggressively probing other systems. Use authorized logs and management systems to look for the same accounts, hashes, destinations, service names, job names, SSH key fingerprints, and time-correlated logons. The objective is scope, not retaliation. [D&R-13](#) carries the lateral-movement detection logic.

Evidence preservation. For a low-risk workstation, screenshots, command output, hashes, and exported logs may be enough. For legal, regulatory, or high-impact incidents, preserve according to formal forensic procedure and do not alter original media when an image is required. Capture live process context - command line, loaded modules, open files, working directory, sockets, parent process - before terminating a suspicious process, because that context does not survive the kill.

The recovery decision.

Situation	Recommended direction
Single unwanted file, never executed, quarantined	Remove source copies, scan, verify; rebuild usually unnecessary
Known malware executed as a standard user, bounded persistence	Contain, scope, eradicate, rotate exposed credentials, validate
Confirmed administrator or root compromise	Strongly consider rebuild; rotate privileged credentials; inspect peers
Unknown kernel or boot-level tampering	Rebuild from trusted media; preserve forensic evidence
Destructive or encryption activity	Immediate isolation, activate the incident plan, preserve evidence, recover from trusted backups (D&R-18)
Cannot explain persistence, or repeated reinfection	Escalate and rebuild rather than repeatedly cleaning symptoms

Cleaning a machine is not the same as proving it trustworthy. When trust cannot be re-established after a privileged compromise, rebuilding from trusted media is the honest technical answer.

Validation after eradication. The incident is not over when the file is gone.

1. Reboot at least once when operationally safe, then repeat the connection, process, persistence, and account checks.
2. Confirm the suspicious destination no longer receives traffic from the implicated process or any replacement.
3. Confirm security controls are enabled and current.
4. Run an appropriate endpoint scan.
5. Verify patched versions and hardened configuration at the original entry point.
6. Review identity logs for post-cleanup access using old credentials.

7. Monitor for recurrence over a window proportionate to the threat and the system's role.

The live-incident decision tree.

1. Destructive activity, confirmed remote shell, credential theft, or a strongly malicious process? If yes: isolate and preserve. If no: continue triage.
2. Can you map the suspicious network activity to a process and executable? If no: improve visibility. If yes: validate the binary, parent, command line, and persistence.
3. Is there execution evidence, or only a suspicious file? If only a file: determine whether it executed before expanding scope. If executed: build the timeline.
4. Was privilege obtained? If administrator, root, or sensitive credentials were exposed: expand scope and favour rebuild plus credential rotation.
5. Do you know the entry point and the persistence? If not: do not declare the system clean.
6. After eradication, does the activity return after reboot and monitoring? If yes: assume missed persistence, a missed credential path, or a reinfection source.

Suspicion scoring without panic. Add confidence when independent signals agree; subtract it when an ordinary explanation fits.

Observation	Typical weight
Geolocated address only	Very low
Unsigned binary in a user-writable path	Moderate
Unknown persistence launching that same binary	High
Process tree consistent with remote execution	High
Known-malicious indicator plus matching process behavior	High
Credential theft or security-control tampering evidence	Very high
Activity disappears only when a legitimate VPN exit changes	Supports the network explanation, not command and control

Closing doctrine. Good incident response is disciplined scepticism. Do not accept the first frightening explanation, and do not accept the first reassuring one either. Build the story from independent evidence. When the evidence points to a VPN exit, a legitimate updater, or a non-executed unwanted file, say that clearly too. The practice is not about finding an adversary in every anomaly; it is about finding the truth quickly enough to act.

33. Mobile compromise response

A suspected mobile device is a potentially hostile sensor and an identity token until cleared. It can expose conversations, messages, location, contacts, calendars, authentication prompts, cloud sessions, and instructions issued in the owner's name. End-to-end encryption protects data in transit; it does not protect a message once a compromised endpoint can read the screen, microphone, keyboard, or notification stream.

Organization-specific emergency, classified-spill, communications-security, counterintelligence, legal, privacy, records, and evidence procedures control. This section does not bypass them.

The decision rule: you do not need proof before taking temporary protective action. The objective is continuity of command, protection of people and information, and evidence-preserving containment.

The first sixty seconds, for the device holder.

1. **Stop sensitive use.** Do not call, message, authenticate, browse, or discuss the suspicion near the device.

2. **Move it out of the decision space.** Screen-up on a clear surface outside sensitive discussions. Maintain physical control. Do not carry it into another sensitive meeting, vehicle, residence, or operations center.
3. **Report from a known-good channel.** Contact security or the duty officer - not from the suspect device.
4. **Preserve it.** Do not unlock, reboot, update, reset, delete, uninstall, scan, remove the SIM, connect a cable, or inspect settings unless the incident lead directs it.
5. **Shift operations.** Move to an approved alternate device and channel, and tell a small need-to-know group that instructions from the old number or accounts require independent callback verification.

Never, on your own: factory-reset the device; remote-wipe it; install a security application; browse for indicators; forward suspicious links from it; place it in an improvised or untested shielding container; destroy it; or let an unvetted technician handle it.

A known-good device is security-controlled, fully updated, and not paired or synchronised with the suspect device. It must not depend on the same potentially compromised session, browser profile, password-manager unlock, messaging link, or recovery factor until those are confirmed trustworthy.

Containment is a command decision, not a ritual. Power loss, management changes, network interaction, SIM handling, incorrect passcode attempts, and peripherals can all affect evidence. Airplane mode, shutdown, and radio isolation each trade remote-risk reduction against evidence and access.

Situation	Preferred action	Risk to control
Device already powered off	Leave it off; secure it; report the observed state	Do not charge, boot, or unlock unless the response team directs
Trained team arriving in minutes	Leave it on and untouched; move it outside the sensitive space; keep line of sight	Do not unlock; do not connect power or accessories
Validated radio isolation and a trained handler available	Use the tested isolation system with the battery and transport procedure	Consumer pouches leak; a device searching for signal drains fast
Active recording, exfiltration, or safety risk outweighs volatile evidence	Authorized leader documents the time and powers it off; does not restart it	May lose memory evidence and delay acquisition
Desire to remote-wipe "just in case"	Do not wipe. Preserve cloud, management, identity, carrier, and device evidence first	A wipe destroys the strongest evidence and may not remediate accounts or carrier control

Six hypotheses to test. Endpoint exploitation or mercenary spyware; a malicious or abused application, profile, certificate, accessibility service, or device-administration grant; cloud or identity account takeover; carrier, SIM, or number-port compromise; telecommunications interception; and benign anomaly or false positive. Each has its own evidence sources, and each has an assumption that must not be made - a clean consumer scan does not rule out the first, an unfamiliar name alone does not prove the second, the handset may be technically clean in the third, changing the handset does not fix the fourth, and one strange symptom should not become attribution in the sixth.

High-confidence triggers. A verified vendor threat notification; a security-team or vendor finding tied to a specific device, account, exploit, profile, certificate, token, or session; a confirmed unauthorized carrier event, number port, SIM change, or forwarding rule; a command, payment, personnel action, or message sent from the owner's identity that they did not issue; or evidence the device was present during protected, privileged, or operationally sensitive activity.

Weak signals that do not prove compromise alone. Battery drain, heat, slow performance, random reboots, storage pressure, sensor indicators, or an application crash; a foreign address, unusual geolocation, poor call quality, spam, or a single security-tool alert; different behavior after an operating-system update, travel, roaming, a new VPN, or a new management policy.

Preserve remote evidence in parallel with device custody, from known-good administrative systems, before logs roll over: identity provider sign-ins and token events; mailbox audit, forwarding, inbox rules, delegate access, and OAuth grants; device-management enrolment, compliance, and command history; platform account device lists, security events, and recovery changes; carrier SIM, port, and forwarding events; messaging linked sessions and key changes; network authentication, VPN, DNS, and wireless association logs; and physical access, visitor, and repair records.

Actions normally prohibited before approval - because each destroys evidence or reduces options: unlocking and browsing; rebooting or powering off; updating the operating system or applications; deleting an application, profile, or certificate; running consumer scanning tools; removing or swapping the SIM; factory reset or remote wipe; and connecting a cable, charger, computer, or accessory.

Rotate access from a known-good device, in priority order. Identity, email, and single sign-on first - credentials, sessions, tokens, unknown devices, keys, recovery methods, delegates, and forwarding. Then the password manager. Then the carrier: account PIN, port-out lock, and SIM history. Then messaging and collaboration linked devices and sessions. Then cloud and business systems tied to fiduciary, personnel, or operational authority. Then financial and personal high-impact accounts. Do not send reset codes to the suspect device, and do not enrol a new factor from a suspect browser session.

Counter-impersonation is the control that protects the organization while the device is out. Require callback to a pre-established number for material decisions; use a verifier or challenge that is not stored on the suspect device; temporarily suspend single-person approval for payments, public statements, access grants, and personnel actions; and tell recipients not to trust a familiar writing style, voice note, video, contact card, caller identifier, or existing message thread by itself.

Recovery tiers.

Tier	Use when	Minimum recovery
A - Explained, low residual risk	Evidence supports a benign cause; no meaningful custody, identity, or carrier exposure remains	Correct the cause, update, validate accounts and carrier, document, monitor
B - Credible targeted event, or inconclusive with high consequence	A targeted lure, unknown access, custody gap, or session anomaly leaves material doubt	New or freshly managed device, clean enrolment, new credentials and sessions, carrier review, selective restore, enhanced monitoring
C - Confirmed or critical compromise	Endpoint, account, carrier, or impersonation compromise confirmed or probable, or consequence demands zero residual trust	New hardware and SIM as directed; full identity and session reset; key replacement; adjacent-account review; forensic retention of old equipment

Clean provisioning. Current supported hardware from trusted inventory; enrolment through management before sensitive use; credentials and phishing-resistant factors created from known-good systems with all prior sessions revoked; only required applications from official or managed distribution; **no default full-image restore from the suspect device** - restore selected content only after the incident team evaluates risk and business need; messaging groups and critical contacts re-verified through an independent channel; and the suspect device, SIM records, and acquisition artifacts retained under evidence procedures.

No self-clearance. The device holder does not return a device or identity to service based on personal observation or a single clean scan. No antivirus, mobile threat defense, app-store check, or forensic tool can prove the absence of every sophisticated compromise. Clearance is a risk decision supported by evidence, scope, and recovery controls, and the designated authorising official owns it.

High-risk travel posture. Before travel: issue a travel device with minimal data and applications, current patches, phishing-resistant factors, and carrier port and SIM controls; remove unnecessary documents, message history, credentials, and synchronised services; store backup security keys separately from the device; and brief the traveller on custody, inspection reporting, suspicious prompts, charging, pairing, and alternate communications. During travel: maintain continuous custody and report any loss of sight, inspection, repair, unlock request, seizure, swap, or unexpected restart; do not pair with rental vehicles, hotel systems, conference equipment, kiosks, or unknown accessories; use approved chargers and avoid data connections to public charging; and keep devices and wearables out of sensitive meetings unless specifically authorized. A phone outside the room is safer than a muted phone on the table.

34. Wireless and perimeter defense

Wireless is the medium where the perimeter is not where the wall is. The defensive mission is to inventory your own wireless environment, detect unauthorized access points, find weak configurations you are responsible for, and measure how far your signal extends past the physical boundary.

This is passive, defensive work. Observe broadcast metadata - network name, hardware identifier, channel, signal strength, advertised security - never the contents of anyone's traffic. Do not deauthenticate, jam, connect to, or interfere with networks you do not own; those acts are frequently unlawful even during a test. Run it only against networks you own or are explicitly authorized in writing to assess, within a signed scope. If unsure whether an activity is in scope, observe less, not more.

Detection is comparison, so build the baseline first. A wireless inventory is the known-good list of access points your organization operates in a location: their hardware identifiers, network names, channels, expected security, physical placement, and owner. Anchor it on the **hardware identifier**, not the network name - names are freely chosen and frequently duplicated, and keying on them produces confident nonsense.

```
bssid,ssid,expected_channel,expected_security,location,owner,status
```

Confirm ownership of each access point before marking it authorized; do not assume. Record expected channel and security so misconfiguration shows up as drift. Add new access points as they are deployed and retire decommissioned ones, so a returning identifier becomes a flag.

The diff is the signal.

Diff result	Meaning	Typical action
In baseline, unchanged	Expected device, expected configuration	None
In baseline, changed	Configuration drift (channel or security)	Investigate misconfiguration
Not in baseline	Unknown device in your space	Triage as a potential rogue
In baseline, missing	Expected device not seen	Check coverage or outage

Classify before you escalate. Not everything unknown is hostile: authorized (on the inventory); unauthorized internal (on your network but unsanctioned - shadow IT or rogue); neighbour or transient (legitimately someone else's, bleeding into your area); and unknown (not yet classified). Neighbouring networks are context, not targets.

The highest-value detection signals.

- A familiar network name paired with an unfamiliar hardware identifier - the impersonation signature, often with a stronger-than-usual signal near a particular location.
- An open or legacy-encryption network inside your space.

- An unexpected channel or security generation for a known network - misconfiguration or impersonation.
- Consumer-grade equipment where enterprise equipment is expected.

Weak and misconfigured networks you own are findings in their own right. Open networks expose traffic and should not exist on corporate space; broken legacy encryption requires immediate remediation; deprecated cipher suites warrant an upgrade recommendation; and missing management- frame protection on capable networks leaves management frames less protected.

Triage, then escalate - never remediate unilaterally. Confirm the observation across multiple sightings to rule out an artifact; classify against the inventory; prioritize by risk, with impersonation of corporate names and open networks ranking highest; document identifier, name, security, channel, signal, location, and time; and escalate to the engagement lead. Do not deauthenticate, jam, or connect to a suspected rogue to "prove" it. Those actions can be unlawful and disrupt legitimate users. Observe, document, escalate.

Perimeter leakage. A perimeter assessment compares where the property ends with where the signal can still be heard. Some leakage is unavoidable; the goal is to understand and minimize it. Walk the boundary and just outside it with location capture running, record where each authorized network remains detectable and at what signal, and find the fade point for each. Stay on lawful ground: conduct the assessment from your own property and adjacent public space, and measure *your own* leakage rather than surveying others.

Signal strength is a rough proxy for distance, not a locator. It varies with device, antenna, orientation, and environment - walls and reflections distort it badly. Use it for relative comparison along a walk, corroborate with multiple readings, and treat location estimates as approximate.

Typical findings map to concrete recommendations: strong signal far past the boundary -> reduce transmit power; leakage on one side only -> directional antennas or reposition; an edge device bleeding outward -> relocate inward; a sensitive network name broadcast widely -> reconsider naming and broadcast scope.

Data provenance for wireless captures. Geo-tagged wireless data ties devices to places and is sensitive. Every observation record carries: timestamp in UTC, hardware identifier, network name, channel, band, advertised security, signal, position, run identifier, and operator. Every run carries a manifest with the authorisation reference, site, device, and start and stop times. Collapse observations by hardware identifier, keeping first-seen, last-seen, observation count, best signal, and best-fix position - the count and time span are evidence of persistence. Hash the artifacts (Section 21.16), store captures encrypted, apply a retention policy, and dispose per Section 23.

Field safety is not optional. The operator never drives and the driver never operates. Plan routes, share the plan, obey traffic law, do not trespass, and mount equipment securely. No dataset is worth an accident.

The wider physical layer. TX-01 extends this to the full radio-frequency picture - cellular, short-range, satellite navigation spoofing, sub-GHz implants, jamming, and backhaul - with the sensing-tier model that tells you what your instrumentation can and cannot see. TX-02 covers optical media and the fiber-tapping surface where bulk capture actually happens. Both state the structural limit that governs this whole domain: **passive interception is undetectable in principle**, so the control is cryptographic, not telemetric, and an empty alert queue for an uninstrumented band or span is a coverage gap rather than a clean result.

Part V - Program Operations

Part IV covered the work. Part V covers the program around it: how teams test each other safely, how AI fits into operator work without degrading it, how the post-quantum transition lands on an operator's desk, and how authorized testing against someone else's systems is conducted.

35. The purple-team loop

Team-based engagements split the work across four roles. Red emulates an authorized adversary; Blue detects, triages, and responds; Purple runs the feedback loop that turns red findings into blue detections; White controls the exercise, keeps it safe, and scores it.

Every pattern here assumes a signed scope, written authorisation, and an owned or explicitly permitted range. Nothing in this section provides exploits, payloads, command-and-control, or evasion; the red-team material is **engagement scaffolding and guardrails**, not offensive tooling.

Red's discipline is scope control, not exploitation. In a responsible program the value is not in the technique; it is in disciplined scope control, deconfliction, and evidence, so Blue and White can learn from the activity.

Scope guard - fail closed. Wrap engagement tooling so it refuses to act on any host, range, or domain that is not on the signed, hash-pinned allow-list. Unknown target means stop.

```
#!/usr/bin/env bash
# scope_guard.sh : refuse any target not on the signed, hash-pinned allow-list.
set -Eeuo pipefail
scope="{SCOPE_FILE:?signed scope file required}"
want_sha="{SCOPE_SHA256:?expected scope hash required}"
got_sha="{sha256sum "$scope" | awk '{print $1}'}"
[[ "$got_sha" == "$want_sha" ]] || { echo "scope hash mismatch - refusing" >&2; exit 3; }

target="{1:?target required}"
if grep -Fxq -- "$target" "$scope"; then
    echo "$(date -Is) target=$target verdict=in-scope op=$USER" >> engagement.log
else
    echo "$(date -Is) target=$target verdict=OUT-OF-SCOPE op=$USER" >> engagement.log
    echo "refusing: $target is not in signed scope" >&2
    exit 4
fi
```

Three properties make it trustworthy: it fails closed, so an unlisted target is refused rather than allowed; it verifies the scope file's integrity against the hash recorded in the rules of engagement, so a file edited after signing is rejected; and it uses exact and CIDR matching rather than substring matching, so a look-alike host cannot slip through. Log allow decisions as well as denials - logging only denials leaves allowed actions with no audit trail.

Deconfliction is what separates exercise traffic from a real incident. Maintain an append-only record of the authorisation window, allowed activity classes, contacts, and a deconfliction signal. Refuse to proceed outside the authorized time window. Emit a marker - ticket identifier, source host, activity class, operator - to the agreed channel for every action, and write a closing entry on stop or abort. Silent exercise activity that Blue mistakes for a genuine intrusion burns real incident-response cycles and destroys trust in the program.

Evidence capture without over-collection. Record the exact command, the operator, and an ISO 8601 timestamp; hash artifacts so tampering is detectable; redact obvious personal data on the way in; write to an access-controlled directory with restrictive permissions; and produce a manifest the report writer can verify independently. Capture failed steps too - capturing only successes biases the report.

Purple closes the gap between what Red did and what Blue saw.

Build a coverage matrix that pairs each technique exercised with whether a detection fired. Join on **technique identifier and time window**, never on free-text names, and never count a detection that fired outside the exercise window.

```
#!/usr/bin/env python3
"""coverage_matrix.py : pair exercised techniques with detections that fired."""
import json, sys

exercised = json.load(open(sys.argv[1])) # ["T1110.003", "T1053", ...]
detections = json.load(open(sys.argv[2])) # [{"technique": "T1110.003", ...}]
```

```
fired = [d["technique"] for d in detections]
rows = [(t, "covered" if t in fired else "GAP") for t in sorted(exercised)]
for t, status in rows:
    print(f"{t:12} {status}")
cov = sum(1 for _, s in rows if s == "covered") / max(len(rows), 1)
print(f"coverage={cov:.0%}")
```

Report the explicit gap list, not just a percentage. A coverage number with no gaps enumerated is a metric, not a finding.

A replay harness gates detection changes. Store a small, benign, version-controlled telemetry sample per technique, replay it against a **test** detection pipeline isolated from real response, and compare against the expected outcome. A regression makes the run exit non-zero. Two rules are absolute: replay never touches the live response pipeline - paging on-call with a test is its own incident - and samples must be benign recorded telemetry containing no live or sensitive data.

Track gaps to closure. One item per gap with technique, description, owner, and due date; linked to the proposed detection or hardening change; status tracked through open, in-progress, retest-pending, closed; and **closure gated on a passing replay test**. Report aging open items. Without the owner and the retest gate, the same gaps are rediscovered at the next exercise.

36. Exercise control and deconfliction

White runs the exercise: scheduling injects, holding the deconfliction registry, scoring the event, and stopping it instantly when needed. Safety and the abort path come first.

The abort broadcast is the most important control in the room. One authoritative channel every team subscribes to. On abort, write to the immutable exercise log *first*, then broadcast a clear stop with reason, time, and caller. Track acknowledgement per team until all confirm. Do not allow the exercise to resume without an explicit, logged restart.

```
#!/usr/bin/env bash
# abort.sh : halt all exercise activity and notify every team.
set -Eeuo pipefail
reason="${1:?abort reason required}"
msg="STOP EXERCISE reason='$reason' by=$USER at=$(date -Is)"
echo "$msg" >> exercise.log           # log first
for team in red blue purple white; do
    printf '%s\n' "$msg" >> "channels/$team.abort"
done
echo 'abort broadcast sent; awaiting acknowledgements'
```

Test the abort path in the dry run. An abort path that has never been exercised is a hypothesis, and discovering it is broken during a real safety event is the worst possible time.

Inject scheduling. Load the approved plan; refuse to fire when a global pause or abort flag is set; release each inject at its scheduled offset; log the **actual** release time and recipient, not just the planned one; and skip and flag any inject whose preconditions are not met rather than forcing it.

Scoring must be fixed in advance and traceable to the log. Agree objectives and point values before the exercise; derive every score from logged, timestamped events; attach the supporting log references to each scored objective; and emit an after-action summary with scores, gaps, and next steps. Scoring from memory after the fact, or adjusting the rubric once results are known, converts an exercise into an opinion.

37. AI-assisted operations

Frontier AI is an accomplice, not an autopilot. Used well it compresses the tedious parts of security work and sharpens reports. Used carelessly it produces confident, professional-looking nonsense that wastes triagers' time and burns reputation.

The slop problem is real and measurable. Through 2025 and into 2026, vulnerability programs were flooded with generated reports that read convincingly but describe issues that do not exist, cite functions that are not real, and cannot be reproduced. At least one major open-source project reported roughly a fifth of its submissions in that period were generated slop while genuine ones fell sharply, and paused its bounty program before resuming once quality recovered. The same stretch also produced genuine, novel vulnerabilities found by skilled humans using AI and disclosed responsibly. AI collapsed the median report quality while raising the ceiling. Which side of that split you land on is a choice you make on every submission.

Where it genuinely helps.

- **Understanding fast.** Explaining unfamiliar code, frameworks, error messages, or an obscure protocol so you can work intelligently.
- **Idea generation.** Brainstorming test cases and edge conditions, which you then verify by hand.
- **Reading the rules.** Summarizing long policies so you do not miss a scope or exclusion clause - then confirming against the source.
- **Report craft.** Tightening your own verified findings into clear, well-structured prose.
- **Safe tooling.** Writing workflow and hygiene scripts: scope checks, loggers, trackers, normalizers.

Where it hurts.

- **Hallucinated findings.** Models invent plausible-sounding vulnerabilities, identifiers, endpoints, and parameters. Never trust an AI-asserted defect you have not reproduced.
- **False confidence on severity.** A model will happily call a low-impact issue critical. Score from evidence.
- **Scope blindness.** The model does not know the authorisation boundary. That judgment stays human, every time.
- **Data leakage.** Never paste real user data, secrets, tokens, client records, incident evidence, or another person's information into any AI tool.

The human-in-the-loop standard, adopted in full.

1. A human reviews, understands, and can explain every piece of AI-assisted content before it is used or submitted.
2. Reproduce the finding yourself, manually, before reporting it.
3. Disclose substantial AI assistance when a program asks.
4. No autonomous agents acting against targets, opening changes, or submitting reports on their own.
5. The quality bar does not move because AI was involved. You remain fully responsible.

Prompting that works for security. Give the model context and your own observations; ask it to explain or critique rather than to invent findings. Ask it to argue *against* your hypothesis - "why might this not be exploitable?" catches false positives early. Keep an iterated prompt library for recurring tasks. Treat every factual claim - an identifier, an API behavior, a configuration default - as unverified until confirmed against a primary source or a live test. PB-28 is the GreyNOC operator prompt library: P-DEF, P-OFF, P-BTY, P-BUG, and P-REP families, each entry carrying a use case, required inputs, the prompt, an output contract, and the verification step that must pass before the output is used. Note the boundary: a P- identifier names *how the analyst worked* and is never cited as a finding class.

Coding assistants need repository-level controls. An assistant performs better when the repository gives it what a capable engineer needs: project context, architecture rules, coding standards, security boundaries, validation commands, and a definition of done. Instruction files in the repository, validation scripts that prove the work passes, and human review that decides what merges - in that order.

The rule that matters most: **do not measure an assistant by how fast it writes code; measure it by whether it produces small, understandable, validated changes a human can safely review.**

Level	The assistant can	Human control
Explain	Read and explain	No file edits
Draft	Suggest code or patches	Human applies changes
Edit	Modify approved files locally	Human reviews the diff before commit
Validate	Edit and run commands	Validation logs required
Change proposal	Open a change for review	Protected branches and required review
Scheduled automation	Run event-driven tasks	Strict allowlists, gates, and a rollback plan

Require human approval before deleting files, running destructive commands, changing authentication or authorisation, adding dependencies, modifying data migrations, changing deployment settings, touching production infrastructure, or disabling tests and security checks.

Security triage for generated code. Review every generated diff through a security lens before merge: credential-looking strings; authentication or session logic changed to make a test pass; authorisation checks simplified or bypassed; raw input reaching a shell, query, path, or template; sensitive data in logs; dependencies added without justification; and internal stack traces exposed to users. The recurring failure modes are scope creep, shallow tests that only mirror the implementation, silent weakening of a security control, dependency sprawl, over-refactoring, and confidently invented APIs.

When generated code causes a problem, handle it like any other engineering incident - and do not hide that AI assisted the change. Pause related automation; contain by revert, flag, or rollback; collect the change, the task, the validation logs, and the review comments; classify the failure as bad task, bad context, missing test, weak review, ignored instruction, or a missed approval; fix the product issue; then fix the *workflow* issue that let it through.

Defending AI systems you operate is a distinct discipline. PB-06 covers AI-augmented detection and guardrails; PB-15 through PB-20 cover the authorized-testing side with defect-class catalogs (L, S, G, I); PB-19 covers model behavior, which routes to a model-safety channel rather than a security queue; PB-20 and D&R-20 cover inference-serving and multi-tenant isolation; D&R-09 covers adversary automation and abuse of AI features you own; D&R-19 covers AI-specific incident response and evidence; and PB-21 through PB-24 carry the governance, development, data, and resilience controls. Two operator-level rules generalise from that body of work: treat model artifacts obtained from any untrusted source as untrusted code - never load, unpickle, or execute them, inspect statically and offline - and remember that inference is metered, so consumption controls are a real availability and cost control, not an afterthought.

38. Post-quantum readiness for operators

The post-quantum transition is not a future project; it is an inventory problem, a migration-defect problem, and a capacity problem that lands on operators now.

Why now, before any quantum computer breaks anything. Harvest-now-decrypt-later: an adversary with the budget to record encrypted traffic can store ciphertext today and decrypt it years later. For anything whose confidentiality has a long shelf life, the clock started the day the traffic was first recorded, not the day a capable machine boots. PB-02 covers exposure assessment; TX-02 covers where bulk optical capture physically happens.

Naming discipline is a house rule. Use the standardized names - **ML-KEM**, **ML-DSA**, **SLH-DSA** - in every report, ticket, and finding. Submission names (Kyber, Dilithium, SPHINCS+) are acceptable only as parenthetical aliases. For Falcon, write *Falcon / expected FN-DSA (FIPS 206 in development)* until FIPS 206 is final. Never represent a selected algorithm, a draft, or a reserved standard name as a completed compliance target. The full reference is [CONVENTIONS Section 1](#).

The operator's four jobs.

1. **Inventory.** You cannot migrate cryptography you have not found. A cryptographic bill of materials records where each algorithm is used, by what, with what key lifetime, and how hard it would be to change. PB-01 covers discovery and crypto-agility scoring. The cheapest starting telemetry is the handshake probe from [Section 21.3](#) extended to record the negotiated group.
2. **Prioritize by data shelf life.** Rank by how long the confidentiality must hold, not by how interesting the system is. Long-retention data, long-lived credentials, and archives outrank ephemeral session traffic.
3. **Prefer hybrid during transition.** Classical plus post-quantum run together means an adversary must break both. Watch for the named groups in [CONVENTIONS Section 2](#) in handshakes, and treat a negotiation that silently drops back to classical as a finding, not a quirk - PB-03 covers downgrade and middlebox tampering.
4. **Watch for migration defects.** The dominant risk class right now is not a broken primitive; it is defects introduced while swapping primitives under deadline. PB-08 catalogs them as C1 through C15, PB-13 covers agility architecture and rollback gates, and PB-27 covers the capacity and interoperability consequences - larger keys and signatures have real network, HSM, and storage costs.

Where operators most often meet this. Transport and tunnels (PB-09); certificate lifecycle and revocation at post-quantum scale (PB-10); code signing and firmware, where stateful hash-based signatures carry a genuine operational hazard if key state is mishandled (PB-11); token and signature integrity, including algorithm confusion (PB-05); key management and data-at-rest rewrap (PB-25); and enterprise identity, document signing, and relying-party migration (PB-26).

Symmetric cryptography needs attention too. Grover's algorithm halves effective symmetric strength, so for long-retention data prefer AES-256 and SHA-384 over AES-128 and SHA-256.

Two honest limits. First, post-quantum *authentication* at scale is not solved by current deployed messaging constructions - they still authenticate with classical primitives, and key-transparency systems are the present mitigation (PB-04). Second, validation of an implementation is not assurance against physical attack: TX-03 documents the side-channel assurance gap, and the compiler-induced timing defects that can undo a constant-time implementation in silicon.

Personal and small-scale sovereignty is now achievable, with caveats. Standardized primitives shipping as ordinary dependencies, commodity hardware with real capacity, and zero-configuration tunnelling have together put a hybrid post-quantum, end-to-end-encrypted relay within reach of one person. That is genuinely new. It is also worth stating the limits precisely, because a revolution oversold is a revolution discredited: end-to-end encryption hides content, not metadata; a centralized tunnel that provides reachability sees connection metadata even when it cannot see message content; endpoint compromise defeats the entire construction; trust-on-first-use only closes the first-contact gap if people actually compare the verification value; and a device on a nightstand has the availability of a device on a nightstand. Name those limits in any assessment that touches this pattern.

39. Authorized testing and coordinated disclosure

Authorized offensive work is in scope for GreyNOC operators, and it is governed more tightly than any other activity in this handbook. **CONVENTIONS** [Section 6](#) (rules of engagement), [Section 7](#) (the AI-system addendum), and [Section 8](#) (defect-class registry and evidence notation) are mandatory and bind every bug-bounty playbook. This section is the operator's practice around them.

Three rules above everything. No program reward justifies bending them.

1. **Authorisation first.** Only test where you have explicit, in-scope, written permission. The program page is the authorisation boundary, not your assumptions.
2. **Minimum impact.** Prove the issue with the smallest possible footprint, then stop. Never access, copy, modify, or destroy real data.
3. **Report privately.** Disclose through the approved channel only. Public disclosure happens when, and if, the platform and program allow it.

Before joining any platform. Test only in-scope assets. Do not scan, fuzz, brute force, spam, phish, social-engineer, or attempt denial of service unless the program explicitly allows it. Stop testing the moment impact is proven. Keep clean notes: dates, endpoints, parameters, steps, evidence, and impact.

Separate the hat from the wallet. If a finding was discovered on GreyNOC time, on GreyNOC infrastructure, or under a client engagement, route the reward question through leadership before accepting a personal payout. Authorized client testing and personal bounty hunting are different activities and stay cleanly separated. Employment agreements can also affect whether an individual may personally claim a reward for findings made with work resources.

Safe harbour and the authorized channel. A program's safe-harbour statement is its commitment not to pursue good-faith researchers who stay within published rules. Strong language says the organization will not bring claims for authorized good-faith research, will not refer you for prosecution, and considers the activity authorized under applicable computer-misuse law. Weak or missing safe harbour means treat scope even more conservatively - and when in doubt, ask rather than test. Safe harbour covers in-scope, good-faith activity only; it does not cover out-of-scope testing, data exfiltration, or extortion.

Where no program is published, many organizations still name a reporting contact and policy in a `security.txt` file (RFC 9116) at `/.well-known/security.txt`. Checking it is the polite, correct way to find where an organization wants to hear about issues.

Operator identity. A dedicated research identity keeps the work separate from personal accounts and builds triager trust: a professional handle, the approved operator email when reporting on GreyNOC's behalf, multi-factor authentication on every platform without exception, and a short, accurate bio. Prepare the kit too - an intercepting proxy, a separate hardened browser profile with no personal logins or extensions, evidence capture tools, notes and report templates, and a password manager. Use a VPN only where the program allows it; some programs restrict or geofence it.

Recon before testing is reading, not attacking. Read the full policy, scope, and exclusions twice. Browse the application as an ordinary user and map the authentication, registration, search, upload, profile, password-reset, and visible API surfaces. Note the technology stack from public signals without aggressive probing. Identify which features touch *your own* data, so access control can be tested safely against accounts you control. Move to active testing slowly and only within the rules, and respect rate limits - if the program forbids automated scanning, that includes most scan-everything tooling.

Two controlled accounts are the safest access-control setup. Register two accounts you fully control, and demonstrate any cross-account claim between *those two*. Never between yours and a real customer's. And stop at proof: one clean cross-account observation is sufficient; iterating through other identifiers to see "how many" you

can reach converts a clean finding into a serious violation.

Severity follows evidence. Most platforms want a severity estimate, and CVSS v4.0 is the current industry standard for it. Provide the base vector, score conservatively, and claim only what you actually proved. Do not claim account takeover unless you safely demonstrated account takeover within scope. Inflated severity is the fastest way to lose triager trust; let the triage team raise it. Severity (how bad if exploited) is also distinct from likelihood (how probable exploitation is) - mature programs pair the two, but your job is to report the issue clearly rather than to run their risk model.

What is commonly not worth reporting unless the program says otherwise or you can demonstrate concrete additional impact: missing security headers with no demonstrated exploit; self-inflicted script execution requiring the victim to paste a payload; clickjacking on pages with no sensitive state-changing action; bare open redirects with no further impact; verbose errors or version banners with no exploit; rate-limiting weaknesses with no concrete impact; scanner output with no manual validation; and issues in third-party services outside the program's scope.

Defect-class discipline. Every bug-bounty playbook in the library numbers its defect classes with a unique prefix so a class identifier is globally unambiguous: **C** for cryptographic implementation and post-quantum migration defects (PB-08), **L** for the LLM application layer (PB-16), **S** for AI supply chain and model artifacts (PB-17), **G** for agentic systems, tools, and the connector boundary (PB-18), and **I** for inference infrastructure and multi-tenant isolation (PB-20). A finding cites exactly one primary class; a chain cites the primary class and lists the others in sequence, because the chain is what carries the impact. **CONVENTIONS Section 8.1** holds the authoritative table and the precedence rules for the cases where the one-line heuristics disagree.

AI targets add non-negotiable rules (**CONVENTIONS Section 7**). Classify security defect versus model behavior *before* submitting - a jailbreak is a security finding only when it produces a concrete security consequence such as an unauthorized tool invocation, access to data you were not entitled to, a privilege change, or code execution. Never generate genuinely harmful content as proof; demonstrate the control failure with an inert canary (**GNBB-CANARY-<uuid4>**), not the payload. Test with your own tenancy. Treat denial-of-wallet as real harm and cap the request budget up front. Record the trial ledger for every stochastic finding. And be identifiable where the program permits it, so defenders do not burn incident-response cycles on you.

The report is the product. A complete report has: a specific title naming the defect type, the location, and the impact; a plain two-to-four-sentence summary; program, platform, and affected asset; vulnerability type and a conservative severity estimate with its vector; explicit scope confirmation; what testing was and was not performed; numbered reproduction steps; a minimal proof of concept; observed and expected results; honest impact; evidence with timestamps; a remediation recommendation; and operator safety notes confirming that no real user data was accessed, no destructive or denial-of-service testing was performed, and no persistence was attempted. **Section 41** carries the template.

Specific titles beat vague ones every time: "Reflected script execution in the **q** parameter on **/search**" tells a triager what to route; "XSS found" does not.

Triage etiquette. Be patient - triage takes time. Answer questions clearly and quickly; responsiveness builds reputation. Do not re-test aggressively after submitting. Keep all communication inside the platform's report thread; off-platform contact with staff, public call-outs, or pressure to disclose can void safe harbour and end your standing. Never threaten disclosure or imply leverage - that is extortion, not research. A duplicate means someone reported it first, not that your work was wrong; an informative outcome means the team acknowledges it but will not act. Both are normal.

A good first goal is not a critical finding. It is one clean, authorized, reproducible, professional report. Process quality is the skill that compounds.

Part VI - Reference

40. Quick-reference card

Designed as a pull-out. Print it and keep it at the console.

Strict-mode openers.

```

# Bash
set -Eeuo pipefail; IFS=$'\n\t'; umask 077
WORK="$(mktemp -d)"; trap 'rm -rf "$WORK"' EXIT
trap 'exit 130' INT; trap 'exit 143' TERM  # signals must exit; EXIT still cleans up
        
```

```

# Python
import argparse, logging, sys
from pathlib import Path
        
```

```

# PowerShell
Set-StrictMode -Version Latest; $ErrorActionPreference = "Stop"
        
```

Safe-by-default reflexes.

Do	Instead of
<code>subprocess.run(..., check=True)</code>	<code>os.system("cmd " + userinput)</code>
Quote <code>"\$var"</code> ; use arrays	Unquoted expansion and word-splitting
<code>os.getenv("KEY")</code>	<code>key = "hardcoded-secret"</code>
<code>SupportsShouldProcess / --dry-run</code>	Acting immediately with no preview
<code>mktemp -d</code> plus a cleanup trap	Hand-named temp files left behind
Resolve, then compare paths	Substring checks on a path or a target

Exit codes. 0 success · 1 general error · 2 usage or bad arguments · 126 not executable · 127 not found · 130 interrupted.

Incident first moves. Timestamp and operator log -> isolate if justified (network, not power) -> volatile network state -> process state -> security-control state. Preserve before you change.

Evidence reflexes. Hash on capture. Work on a copy. Sanitize the copy. Verify the copy. Record the command that produces every claim.

Cleanup and secrets reflexes. Encrypt sensitive data at rest; cryptographic erase by destroying the key. Never put secrets in `argv`; redact before logging; rotate after any exposure. Overwrite tools are unreliable on solid-state and journaling storage. Cleanup on `EXIT`; `INT` and `TERM` handlers exit (130 / 143) so a cancelled run cannot continue against deleted state.

Debug one-liners.

```

echo "DEBUG: x=$x" >&2          # Bash
bash -x script.sh              # trace Bash execution
        
```

```

print(f"DEBUG {x}")            # Python 3.8+
        
```

Confidence words. CONFIRMED · PROBABLE · POSSIBLE · NOT SUPPORTED · INCONCLUSIVE. Never "impossible" or "guaranteed clean."

Citation forms. D&R-NN · PB-NN · TX-NN - never a bare number. MITRE cited as name + ID + version. Stochastic results as successes/trials @ parameters (95% Wilson CI low-high).

41. Standard templates

41.1 Script headers.

```
"""GreyNOC Script -- Name / Purpose / Author / Version / Date / Usage"""
import sys

def main() -> int:
    try:
        # --- logic ---
        return 0
    except Exception as e:
        print(f"ERROR: {e}", file=sys.stderr)
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

```
#!/usr/bin/env bash
# GreyNOC Script -- Name / Purpose / Author / Version / Date / Usage
set -Eeuo pipefail
trap 'rm -rf "${WORK:-}" 2>/dev/null || true' EXIT
trap 'exit 130' INT; trap 'exit 143' TERM
# --- logic ---
exit 0
```

```
<# GreyNOC Script -- Name / Purpose / Author / Version / Date / Usage #>
[CmdletBinding(SupportsShouldProcess)] param()
Set-StrictMode -Version Latest; $ErrorActionPreference = "Stop"
try { <# logic #>; exit 0 } catch { Write-Error "ERROR: $_"; exit 1 }
```

41.2 Field-note entry.

```
Date / time / timezone:
Operator:
Authorized environment and authorization reference:
Purpose:
Commands or actions performed:
Observed result:
Interpretation (marked as interpretation):
Follow-up needed:
Sensitive data captured?   yes / no
Sanitized copy created?   yes / no
```

41.3 Run manifest.

```
{
  "run_id": "R-0920-A",
  "site": "HQ perimeter",
  "authorization_ref": "ENG-2026-0042",
  "operator": "op7",
  "device": "field-03",
  "started": "2026-09-20T13:55:00Z",
  "stopped": "2026-09-20T14:40:00Z",
  "notes": "north and east boundary loop"
}
```

41.4 Incident notes.

```

Incident ID / hostname:
Operator:
Start time and timezone:
Trigger / symptom:
Public egress address (recorded before any change):
Suspicious remote endpoints:
Suspicious processes (PID, path, command line, parent):
Relevant file hashes:
Persistence mechanisms:
Accounts affected:
Containment actions (time, authority):
Evidence exported (paths, hashes):
Eradication actions:
Credentials rotated:
Patch / configuration changes:
Validation results:
Assessment (CONFIRMED / PROBABLE / POSSIBLE / NOT SUPPORTED / INCONCLUSIVE):
Open questions:

```

41.5 Mobile incident record. Complete from a known-good system or on paper - never typed into the suspect device.

```

Incident ID:                      Date / time / timezone:
Principal / role:                  Incident commander:
Trigger (exact words and observation):
Device make / model / number / asset ID:
State when observed (on / off / locked / unlocked):
Visible battery / signal / notifications:
Location and activity underway:
Last known-good time:             First suspected event:
Physical custody gaps / repair / inspection:
Sensitive meetings or conversations in window:
Accounts and linked devices:
Containment action, time, and authority:
Known-good channel issued:
Counter-impersonation notice recipients:
Sessions / credentials / carrier actions:
Evidence preserved and by whom:
Chain-of-custody transfers / seal IDs:
Recovery tier (A / B / C) and basis:
Return-to-service approvals:

```

41.6 Vulnerability report. For a bug-bounty platform or coordinated disclosure. Fill every field.

```

Title:          [Defect type] in [feature/endpoint] allows [impact]
Summary:        2-4 sentences, plainly.
Program:        Platform:
Affected asset / endpoint:
Vulnerability type:
Severity estimate (CVSS v4.0 vector if known):
Defect class (CONVENTIONS Section 8.1, where applicable):
Scope confirmation:  In scope? Type accepted? Method allowed?
Testing performed:  Manual / what was and was NOT done.
Steps to reproduce:
  1.
  2.
  3.
Proof of concept:    (minimal; no data, token, or session theft)
Observed result:     (claim only what was proven)
Expected result:
Impact:
Trial ledger (stochastic findings): successes/trials @ parameters (95% Wilson CI)
Evidence:           request / response / screenshot / timestamp
Remediation recommendation:
Operator safety notes:
  - No real user data accessed.
  - No destructive testing performed.
  - No persistence attempted.

```

- No denial-of-service testing performed.
- Testing stayed within program scope.

AI assistance disclosure (if substantial):
Disclosure status: Private until program approval.

41.7 Assessment report skeleton.

Section	Contents
Scope and authorization	Sites, dates, authorization reference, rules of engagement
Methodology	Platform, tools, coverage, and stated limitations
Findings	Prioritized by risk, each with evidence and a recommendation
Evidence	Identifiers, timestamps, locations, file hashes
Recommendations	Concrete remediation per finding, with an owner
Appendices	Inventory deltas, maps, raw data references

Lead with risk, not volume. Three confirmed high-risk findings explained well beat a hundred raw observations. Tie every finding to a clear, achievable recommendation.

41.8 Exception register row. Every exception - a control left off, a port left open, a policy not applied - carries all six fields or it is not an exception, it is a gap.

ID | Asset/user | Control changed | Business reason | Compensating control | Owner | Expiry | Rollback

42. Operator checklists

42.1 Before any engagement or change.

- Written authorization on file, read, and reachable in the field.
- Scope mapped: in-scope and out-of-scope assets listed explicitly.
- Objective stated in one sentence with a definition of done.
- Data sensitivity identified; handling and retention agreed.
- Rollback documented, with the approver named.
- Deconfliction contact and channel agreed (exercises and testing).
- Evidence store prepared: encrypted, access-controlled, with a manifest.

42.2 Script code review.

- Header states name, purpose, author, version, date, requirements, usage.
- Configuration separated from logic.
- Every external input validated; no substring matching for security decisions.
- No hardcoded secrets; secrets never in `argv`; nothing sensitive logged.
- Strict mode and a cleanup trap; temporary files created privately.
- Dangerous actions opt-in; dry-run available for anything that writes.
- Meaningful exit codes; failures are loud, not silent.
- Idempotent, or it clearly refuses to re-run when that would be unsafe.
- Static analysis clean; tests cover the decision logic.
- Rollback or stop instructions documented next to the script.

42.3 Release gate.

- Static checks and tests pass in CI.
- Secret scan clean; dependencies pinned and reviewed.
- Version bumped; changelog entry written.
- Artifacts checksummed; release directory immutable.
- Rollback documented and exercised at least once.
- Deployment rings and promotion and abort criteria defined.
- Owner sign-off recorded; any exception flagged explicitly rather than shipped around.

42.4 Live incident - Windows.

- Record time, hostname, user, and symptom.
- Capture TCP and UDP sockets, routes, and neighbour cache.
- Map suspicious process identifiers to path, command line, parent, and service.
- Hash and check signatures of suspicious executables.
- Review endpoint-protection detections and status.
- Review startup entries, run keys, scheduled tasks, services, and event subscriptions.
- Review script, process-creation, service-install, and logon events.
- Capture recent executable and script writes in a bounded window.
- Contain confirmed malicious activity without deleting evidence.
- Disable persistence, then quarantine confirmed payloads.
- Rotate exposed credentials and patch the entry point.
- Reboot, rescan, and validate no recurrence.

42.5 Live incident - Linux.

- Record time, hostname, user, and symptom.
- Capture socket state, routes, and neighbours.
- Map suspicious processes to executable, command line, working directory, parent tree, and user.
- Hash and identify suspicious files; verify package ownership where applicable.
- Review service units and timers, scheduled jobs, shell profiles, autostart, and authorized keys.
- Review journal, authentication logs, login history, and privilege escalation activity.
- Inspect temporary and shared-memory paths and web roots for recent payloads.
- Correlate service logs with process start and file creation times.
- Contain network, account, and service activity.
- Disable persistence, quarantine payloads, rotate credentials, patch the entry point.
- Restart as appropriate, rescan, and validate no recurrence.

42.6 Mobile compromise - first actions.

- Stop sensitive use; move the device out of the decision space; maintain custody.
- Report from a known-good channel, never from the suspect device.
- Preserve: no unlock, reboot, update, reset, scan, SIM change, or cable.
- Photograph the device state with an external camera; record on/off/locked state.
- Preserve remote evidence in parallel: identity, mail, management, platform, carrier, network.
- Stand up clean command communications and issue the counter-impersonation notice.
- Rotate access from a known-good device in priority order.

- Choose the recovery tier on consequence; no self-clearance.

42.7 Post-engagement.

- Test artifacts removed from authorized systems and documented as removed.
- Collected data moved into an encrypted container; plaintext working copies deleted.
- Credentials generated, used, or exposed during the engagement rotated.
- Evidence manifest complete and verifiable; chain of custody recorded.
- Retention applied; disposal scheduled per [Section 23](#).
- Report delivered; findings tied to owners and remediation.
- Lessons captured: what worked, what to change in the runbook.

42.8 Operator readiness.

- Authorization and scope current for everything you are working.
- Workstation baseline verified; security floor intact ([Section 6](#), [Section 7](#)).
- Evidence store encrypted and access-controlled.
- Check-in cadence agreed for solo or remote work.
- Cognitive-load self-rating honest and acted on.
- Handoff notes written at end of shift, even on an uneventful one.

43. Library map

The handbook points at the playbooks; this is the index. Cross-reference notation is [CONVENTIONS Section 5 - D&R-NN](#), [PB-NN](#), [TX-NN](#), never a bare number.

Detection & Response.

ID	Playbook	Handbook section
D&R-01	Password Spraying	Section 26, Section 27
D&R-02	Brute Force	Section 21.1, Section 26
D&R-03	Distributed Port Scan	Section 25, Section 27
D&R-04	Credential Stuffing	Section 26
D&R-05	Impossible Travel	Section 26, Section 32
D&R-06	Privilege Escalation	Section 26, Section 30, Section 31
D&R-07	Suspicious PowerShell Execution	Section 30
D&R-08	Malware Beaconing	Section 29
D&R-09	AI / Automated Agent Abuse	Section 37
D&R-10	Coordinated Multi-Stage Attack	Section 28, Section 32
D&R-11	Phishing	Section 26
D&R-12	Business Email Compromise	Section 26, Section 32, Section 33
D&R-13	Lateral Movement	Section 32
D&R-14	Active Directory Credential Theft	Section 26, Section 32
D&R-15	Persistence Mechanisms	Section 21.12, Section 30, Section 31

ID	Playbook	Handbook section
D&R-16	Web Shell	Section 30, Section 31
D&R-17	Data Exfiltration	Section 29, Section 32
D&R-18	Ransomware	Section 21.5, Section 32
D&R-19	AI Security Incident Response & Evidence	Section 37
D&R-20	AI Serving Plane & Multi-Tenant Isolation	Section 25, Section 37

AI · Post-Quantum · E2EE.

ID	Playbook	Handbook section
PB-01	Cryptographic Inventory & PQC Readiness	Section 24, Section 38
PB-02	Harvest-Now-Decrypt-Later Exposure	Section 38
PB-03	Hybrid TLS / KEM Downgrade	Section 38
PB-04	E2EE Messaging Protocol Security	Section 38
PB-05	PQ Signature & Token Integrity	Section 26, Section 38
PB-06	AI-Augmented Detection & Guardrails	Section 37
PB-07	Bug Bounty: PQC/E2EE Methodology	Section 39
PB-08	Bug Bounty: Crypto Implementation Defects (C1-C15)	Section 38, Section 39
PB-09	Post-Quantum VPN, IPsec & SSH	Section 38
PB-10	Post-Quantum PKI & Certificate Lifecycle	Section 38
PB-11	Post-Quantum Code Signing & Firmware	Section 22, Section 38
PB-12	Post-Quantum E2EE for Real-Time Media	Section 38
PB-13	Crypto-Agility & PQ Migration Operations	Section 38
PB-14	Quantum Risk Governance & Compliance	Section 38
PB-15	Bug Bounty: AI & LLM Attack-Surface Methodology	Section 39
PB-16	Bug Bounty: LLM Application-Layer Defects (L)	Section 37, Section 39
PB-17	Bug Bounty: AI Supply Chain & Model Artifacts (S)	Section 37, Section 39
PB-18	Bug Bounty: Agentic Systems, Tools & MCP (G)	Section 26, Section 37, Section 39
PB-19	Bug Bounty: Model Behavior & Safety Boundaries	Section 37, Section 39
PB-20	Bug Bounty: Inference Infrastructure & Isolation (I)	Section 25, Section 37, Section 39
PB-21	AI Governance, Asset Inventory & Third-Party Risk	Section 24, Section 37

ID	Playbook	Handbook section
PB-22	AI Secure Development, TEVV & Change Management	Section 22, Section 37
PB-23	AI Data Governance, Privacy, Retention & DLP	Section 23, Section 37
PB-24	AI Resilience, Continuity & Decommissioning	Section 37
PB-25	PQ Key Management & Data-at-Rest Migration	Section 38
PB-26	PQ Enterprise Identity, Messaging & Trust	Section 26, Section 38
PB-27	PQ Capacity, Interoperability & Vendor Assurance	Section 38
PB-28	AI Prompt Library for Cyber Operations	Section 37

Transmission & Physical Layer.

ID	Playbook	Handbook section
TX-01	RF & Radio-Spectrum Intrusion	Section 34
TX-02	Nanometer-Scale Optical Transmission & Intrusion	Section 34, Section 38
TX-03	PQC at the Nanometer Scale: Silicon & Physical Attack	Section 3, Section 38

Shared reference. `CONVENTIONS.md` - Section 1 algorithms, Section 2 hybrid named groups, Section 3 E2EE protocols, Section 4 MITRE mapping conventions, Section 5 example data and cross-reference notation, Section 6 rules of engagement, Section 7 AI-system testing addendum, Section 8 defect-class registry and evidence notation, Section 9 versioned reference baseline.

44. Glossary

Term	Meaning
Baseline	A curated, owner-confirmed known-good snapshot used to detect later drift
Blast radius	The scope of systems, data, and identities a compromise can reach
Canary	An inert, uniquely identifiable marker planted to prove retrieval, leakage, or propagation
CBOM	Cryptographic bill of materials - where each algorithm is used, by what, with what key lifetime
Coordinated disclosure	Reporting privately and giving the vendor time to fix before public detail
Cryptographic erase	Rendering data unrecoverable by destroying the encryption key
CVSS	Common Vulnerability Scoring System; v4.0 is current. Technical severity, not organizational risk
Deconfliction	The signal and record that lets defenders distinguish exercise activity from a real intrusion

Term	Meaning
Dry run	Executing in a mode that previews changes without making them
Evil twin	An illegitimate access point advertising a legitimate network name
Fail-safe / fail-secure	On error, favouring availability (safe) or favouring denial (secure) - a deliberate choice
HNDL	Harvest-now-decrypt-later: recording ciphertext today to decrypt when capability arrives
Hybrid (crypto)	Classical and post-quantum primitives run together; an adversary must break both
Idempotent	Safe to run more than once, with the same result and no additional harm
Indicator	An address, hash, or pattern associated with observed activity
Known-good device	Security-controlled, updated, and not paired or synchronised with the suspect device
Least privilege	Granting only the access a task strictly requires
ML-DSA	FIPS 204 lattice signature scheme (submission name Dilithium)
ML-KEM	FIPS 203 lattice key-encapsulation mechanism (submission name Kyber)
PMF	Protected Management Frames; required by the current wireless security generation
Provenance	The record of where an artifact came from, when, produced by whom and how
Rogue access point	A device broadcasting in your environment without authorisation
ROE	Rules of engagement: the agreed scope, authorisation, and limits of authorized testing
Safe harbour	A program's commitment not to pursue good-faith, in-scope researchers legally
Scope	The exact assets, methods, and defect types an authorisation permits
<code>security.txt</code>	RFC 9116 file naming an organization's security contact and policy
SLH-DSA	FIPS 205 hash-based signature scheme (submission name SPHINCS+)
Slop	Convincing-looking but false or unreproducible generated content presented as a finding
Telemetry sufficiency	Whether the data needed to detect a behavior actually exists and is retained
Trial ledger	The record of trials, successes, parameters, and interval behind a stochastic finding
VDP	Vulnerability disclosure program - a legal reporting channel, usually unpaid
Wilson interval	The confidence interval used for small-sample success rates; correct at the tails

45. References and further reading

Standards and tools evolve. Confirm the current revision of any referenced publication before relying on it for a control or an audit. Dates below are the version or review point this handbook was written against.

Frameworks and control structures.

- NIST Cybersecurity Framework 2.0 (2024) - Govern, Identify, Protect, Detect, Respond, Recover. The [Section 21](#) catalog maps to these functions.
- NIST SP 800-61 Rev. 3 (April 2025) - incident response recommendations and considerations; the lifecycle [Section 29](#)-[Section 32](#) sits inside.
- NIST SP 800-88 Rev. 2 (effective September 2025) and IEEE 2883-2022 - media sanitization and technique selection; the basis for [Section 23](#).
- NIST SP 800-124 Rev. 2 (2023) - enterprise mobile device management baseline; assumed by [Section 33](#).
- NIST AI Risk Management Framework and the Generative AI Profile (NIST AI 600-1) - the governance baseline behind [PB-21](#) - [PB-24](#).
- CIS Benchmarks - configuration baselines usable as drift references in [Section 21.9](#).

Adversary and defensive taxonomies.

- MITRE ATT&CK - Enterprise v19.2 is this library's pinned baseline, verified 2026-08-09. Mapping rules in [CONVENTIONS Section 4](#).
- MITRE ATLAS - collection 2026.06 is the pinned baseline for AI techniques.
- MITRE D3FEND - defensive countermeasure taxonomy, useful when pairing a detection with a control.

Post-quantum cryptography.

- NIST Post-Quantum Cryptography project - authoritative status for FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), FIPS 206 development, and backup-KEM selection.
- NIST SP 800-227 - key-encapsulation construction and implementation guidance.
- NIST CSWP 39-upd1 (final June 2026) - crypto-agility guidance.
- IETF and IANA TLS registries - current protocol status and code points for hybrid named groups. An Internet-Draft is not an RFC.
- [CONVENTIONS Section 9](#) carries the full versioned reference baseline for this collection.

Application and AI security.

- OWASP - injection and secure coding guidance; the basis for the injection lens in [Section 12](#).
- OWASP Top 10 for LLM Applications and for Agentic Applications - cross-checks for [PB-16](#) and [PB-18](#).
- CISA JCDC AI Cybersecurity Collaboration Playbook - AI incident coordination baseline.

Disclosure and severity.

- FIRST - CVSS v4.0 specification, for the severity discipline in [Section 39](#).
- RFC 9116 - [security.txt](#), for finding the authorized reporting channel.
- OWASP vulnerability disclosure guidance, and the community safe-harbour and disclosure directory projects.

Tooling and language references.

- ShellCheck and a shell style guide - static analysis for [Section 14](#).
- Python documentation - [logging](#), [argparse](#), [subprocess](#), [pathlib](#), [secrets](#).
- PowerShell documentation - approved verbs and [ShouldProcess](#), for [Section 16](#).
- Wireless: the IEEE 802.11 standards family and the current Wi-Fi security generations, for [Section 34](#).

Mobile and high-risk individuals.

- CISA mobile communications best-practice guidance, and national-authority guidance for high-risk individuals - the basis for the escalation and posture material in **Section 33**.
- SWGDE best practices for mobile device evidence collection, preservation, handling, and acquisition - written for trained evidence personnel; **Section 33** translates selected preservation cautions into operator and leader actions and is not an acquisition procedure.
- Platform vendor documentation for threat notifications, high-risk protection modes, account security, and device logging. Menus and feature availability change by model, version, region, and management state - verify before relying on a specific step.

Readiness.

- Occupational health and safety authorities for workplace stress, ergonomics, and hand protection - the basis for **Section 5**. Decision-fatigue material draws on the published self-control and ego-depletion literature, including its registered replication work; treat effect sizes as contested and the operational practice (break, batch, reduce scope) as the durable part.

GreyNOC - detection-engineering-first security operations. Authorization first. Least impact. No fabrication: every finding, indicator, and report artifact must be reproducible from evidence.

SOURCE: CONVENTIONS.md

CONVENTIONS

Shared reference for the GreyNOC AI · PQ · E2EE playbook collection. Cited values reflect the standards landscape as of mid-2026; verify against current NIST / IETF publications before operational use, because this space is still moving.

1. Algorithm reference

Key encapsulation (KEM) - replaces RSA / ECDH key exchange

Standard	Name	Basis	Parameter sets	Notes
FIPS 203	ML-KEM (Kyber)	Module-LWE lattice	512 / 768 / 1024	Primary KEM. ML-KEM-768 ciphertext ~1088 B
In development (final expected 2027)	HQC	Code-based (quasi-cyclic)	TBD	Backup KEM selected Mar 2025; not yet a standard ; diversity hedge vs. lattice break

Signatures - replaces RSA / ECDSA / EdDSA

Standard	Name	Basis	Parameter sets	Notes
FIPS 204	ML-DSA (Dilithium)	Module-LWE/SIS lattice	44 / 65 / 87	ML-DSA-65 is the recommended default. Sig ~3309 B, pubkey ~1952 B
FIPS 205	SLH-DSA (SPHINCS+)	Hash-based	many	Conservative backup; large sigs (~7.8-49.9 KB)
FIPS 206 (in development)	FN-DSA (Falcon)	NTRU lattice + FFT	-	Selected algorithm and expected standard name; do not claim FIPS 206 compliance until final publication
SP 800-208	LMS / XMSS	Hash-based, stateful	-	Use now for firmware/code-signing per NSA guidance

Security levels: L1 ~ AES-128, L3 ~ AES-192, L5 ~ AES-256. Grover halves symmetric strength, so for long-retention data prefer **AES-256** and **SHA-384** over AES-128/SHA-256.

Naming discipline. Use standardized names (ML-KEM, ML-DSA, SLH-DSA) in all reports, tickets, and findings. Submission names (Kyber, Dilithium, SPHINCS+) are acceptable only as parenthetical aliases. For Falcon, write **Falcon / expected FN-DSA (FIPS 206 in development)** until FIPS 206 is final. Never represent a selected algorithm, draft, or reserved standard name as a completed compliance target.

2. Hybrid TLS named groups

Hybrid = classical + PQ KEM run together; an attacker must break **both** to recover the session key. Watch for these in ClientHello / ServerHello supported_groups / key_share :

NamedGroup	Value	Composition	Client key share
X25519MLKEM768	0x11EC	X25519 + ML-KEM-768	~1216 B
SecP256r1MLKEM768	(impl-specific)	P-256 + ML-KEM-768	-
SecP384r1MLKEM1024	(impl-specific)	P-384 + ML-KEM-1024	-

Production support: Chrome/BoringSSL, Firefox, Cloudflare, Akamai (default Feb 2026), AWS, OpenSSL 3.5+ (native), wolfSSL, rustls, Botan. `oqs-provider` (liboqs) is the reference experimentation provider and is **not** FIPS-validated - flag it if seen in a production path.

Quick manual check (read-only inspection of a host you are authorized to test):

```

openssl s_client -connect HOST:443 -groups X25519MLKEM768 -tls1_3 </dev/null 2>/dev/null \
| grep -i "Negotiated\|group"
    
```

3. E2EE protocol reference

System	PQ status	Construction	Notes
Signal PQXDH	PQ initial key establishment (Level 2)	X25519 + ML-KEM, Double Ratchet	Sept 2023; PQ at handshake only
iMessage PQ3	PQ initial + ongoing rekey (Level 3)	hybrid ECDH + ML-KEM, periodic PQ rekey (~every 50 epochs)	Uses signatures over MAC; Secure Enclave signing
MLS (RFC 9420)	classical baseline; PQ ciphersuites emerging	TreeKEM group key agreement	Group E2EE; watch ciphersuite negotiation

Open problem to keep in scope: PQ *authentication* ("Level 4") is not solved at scale by PQXDH or PQ3 - both still authenticate with classical primitives. Key-transparency systems (iMessage Contact Key Verification, WhatsApp Auditable Key Directory) are the current mitigations against key-substitution MITM; monitoring them is a detection opportunity (PB-04).

4. MITRE mapping conventions

- ATT&CK** is referenced as **name + ID + content version**, on the same terms as ATLAS below - e.g. *Weaken Encryption* (T1600 , ATT&CK Enterprise v19.2). **Technique names, technique IDs, and tactic assignments all change between versions**, so none of the three is cited bare, and every playbook carrying an ATT&CK mapping table states the version it was verified against in a *Mapping discipline* note directly under the table.
- The tables in this library are verified against **ATT&CK Enterprise v19.2** (site content v19.2, website v5.0.0), checked against `attack.mitre.org` on **2026-08-09**.
- v19 retired the "Defense Evasion" tactic.** TA0005 was renamed to **Stealth**, and a new **TA0112 Defense Impairment** was split out of it. Concealment stayed on the Stealth side (T1027 , T1070 , T1620 , T1564.008 , T1134 , T1542.001 , T1078); tampering with the controls themselves moved to Defense Impairment (T1553 and its sub-techniques, T1556 , T1484 , T1599 , T1600 , T1601). **Neither substitution is automatic** - see the next two bullets. Any artifact still carrying a "Defense Evasion" tag is mapped against a pre-v19 collection and must be re-mapped.
- v19 renumbered and renamed techniques, not just tactics.** T1562 `Impair Defenses` no longer exists: the concept was elevated to the Defense Impairment *tactic*, and its tool-tampering sub-technique was promoted to the top-level **T1685 Disable or Modify Tools**. T1070.001 `Indicator Removal: Clear Windows Event`

Logs was renumbered and reparented to **T1685.005 Disable or Modify Tools: Clear Windows Event Logs** (parent T1070 Indicator Removal still exists, under Stealth). T1484 was renamed **Domain or Tenant Policy Modification**, and T1484.002 to **Trust Modification**. attack.mitre.org redirects a retired ID to its replacement - that redirect is the authoritative remapping.

- *v19 also removed a tactic from some techniques rather than renaming it. **T1550.001, T1550.002, and T1550.003 are now Lateral Movement only***; rewriting their old "Defense Evasion / Lateral Movement" tag as "Stealth / ..." would be wrong. Verify each technique individually; never sweep a rename across a table.
- A table may cite a **subset** of a technique's tactics - the one operative for that playbook - provided the cited tactic is still assigned in the stated version. Say so in the note so the subset is not later "corrected" into an error.
- ICS techniques (T0xxx) use the ICS tactic set, which is distinct from Enterprise; cite the ICS tactic and re-verify separately.
- **ATLAS** is referenced as **name + ID + collection version** - e.g. *LLM Prompt Injection* (AML.T0051, ATLAS 2026.06). **Both the ID and the name are versioned and both do change**, so neither is cited bare: the collection version is what makes a citation checkable. Any playbook citing ATLAS states the collection version it was written against, and a reader re-maps to whatever their platform carries at deployment time.
- The AI playbooks in this library (PB-15-PB-20) are written against the official [mitre-atlas/atlas-data](#) collection **version 2026.06** (format-version 6.0.0, collection modified-date 2026-05-27, published 2026-06-30). Verified from `dist/v6/ATLAS-2026.06.yaml`: **16 tactics** and **173 techniques** (103 parent techniques + 70 sub-techniques), of which **114 list Agentic AI** and **92 list Generative AI** among their platforms - every Generative AI technique also carries Agentic AI, so 114 is the union, not the sum.
- **2026.06 renamed most ML techniques to AI, and renamed some outright.** Cite the 2026.06 name: AML.T0010 is *AI Supply Chain Compromise*, AML.T0018 is *Manipulate AI Model* (formerly *Backdoor ML Model*), AML.T0053 is *AI Agent Tool Invocation* (formerly *LLM Plugin Compromise*), and AML.T0014 is *Discover AI Model Family* - **not** the older *Discover ML Artifacts*, which is a different concept. Every ATLAS-citing playbook in this library has been re-mapped to the 2026.06 names and states the collection it was written against; treat any artifact that does neither as pre-2026.06 and re-map it before reuse rather than assuming the names match.
- **2026.06 added purpose-built RAG and agentic techniques - map to them instead of forcing an older tag** (Section 4's "say so rather than forcing a tag" cuts both ways). The ones this collection leans on: *RAG Poisoning* (AML.T0070), *False RAG Entry Injection* (AML.T0071), *Data from AI Services: RAG Databases* (AML.T0085.000), *AI Agent Context Poisoning* (AML.T0080, with .000 Memory), *AI Agent Tool Poisoning* (AML.T0110), *Poisoned AI Agent Tool* (AML.T0011.002), *Publish Poisoned Models* (AML.T0058), *AI Supply Chain Rug Pull* (AML.T0109), *LLM Prompt Obfuscation* (AML.T0068), *LLM Prompt Self-Replication* (AML.T0061), and *Cost Harvesting: Agentic Resource Consumption* (AML.T0034.002). In particular, AML.T0031 *Erode AI Model Integrity* is **not** the tag for corpus or memory poisoning - it describes degrading the model itself, and does not apply where the model is untouched.
- ATLAS carries a `maturity` field per technique (Realized, Demonstrated, Feasible; in 2026.06 the split is 64 / 90 / 19). Quote it when severity or plausibility is in question: a technique marked *Feasible* has not been observed in the wild, and a report that implies otherwise is fabricating impact.
- Where no clean mapping exists (much PQ-migration-defect work is pre-ATT&CK), the playbook says so rather than forcing a tag.

5. Example-data conventions

- Network examples use **RFC 5737** (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and **RFC 3849** (2001:db8::/32) documentation address space.

- Identities, domains, and tokens in examples are illustrative and non-attributable.
- Detection logic is **JSON-shaped pseudocode**, not a SIEM query. Translate per platform.
- **Cross-reference notation.** The library holds **three independently numbered collections**, so a bare number is never a valid citation. `PB-NN` always means *this* collection (AI · Post-Quantum · E2EE). A playbook in the Detection & Response collection is written `D&R-NN` - e.g. `D&R-09` is AI / Automated Agent Abuse, while `PB-09` is PQ VPN, IPsec & SSH. A playbook in the Transmission & Physical Layer collection is written `TX-NN` - `TX-01` is RF & radio spectrum, `TX-02` is nanometer-scale optical, `TX-03` is PQC at the nanometer scale in silicon. Never cite a bare number across collections.

6. Rules of engagement (binds all bug-bounty playbooks)

Authorized testing only. GreyNOC operates as the submitting firm under platform programs (e.g., HackerOne / YesWeHack) or direct written engagement.

1. **No activity without scope on file.** A signed SOW or an in-scope, in-policy bug-bounty program is a precondition. Out-of-scope assets, third-party dependencies not named in scope, and chained pivots beyond the authorized boundary are prohibited.
2. **Least-impact testing.** Prefer read-only inspection (handshake/cert/token observation) over anything state-changing. No DoS, no data exfiltration beyond minimal proof, no persistence, no lateral movement unless explicitly authorized.
3. **No fabrication.** Every reported finding must be independently reproducible from the evidence in the report. No speculative severity, no invented impact, no copied CVE text passed off as original analysis. If you cannot reproduce it, it is not a finding.
4. **Data handling.** Capture the minimum proof necessary. Never retain third-party PII, secrets, or message plaintext beyond what the program permits; redact in reports.
5. **Coordinated disclosure.** Follow the program's disclosure timeline. Do not publicize before remediation/authorization.
6. **Stop conditions.** On encountering evidence of a prior real-world compromise, customer PII at rest, or anything outside scope - stop, preserve, and escalate to the program owner.

7. AI-system testing addendum (binds Playbooks 15-20)

Section 6 applies in full. AI targets add failure modes that generic web ROE does not cover, so these rules are additive and non-negotiable for any engagement against a model, an agent, an AI-backed application, or the infrastructure serving them.

1. **Classify before you submit: security defect vs. model behavior.** Most AI programs scope *security* defects - broken authn/authz, injection that crosses a trust boundary, data exposure, tenancy failures, infrastructure compromise - and explicitly route *model behavior* (jailbreaks yielding disallowed content, hallucination, bias, refusal bypass) to a separate model-safety channel. A jailbreak is a **security** finding only when it produces a concrete security consequence: an unauthorized tool invocation, access to data the tester was not entitled to, a privilege change, or code execution. Submitting content-policy bypasses to a security queue is the fastest way to lose program standing.
2. **Never generate genuinely harmful content as proof.** Demonstrate the *control failure*, not the payload. Use an inert canary, a policy-marker string, or a benign no-op tool call. If proving a finding would require producing actually weaponizable material, stop and report the control gap with the boundary evidence already in hand.
3. **Self-scoped tenancy.** Test with your own accounts, tenant, documents, keys, and agents. Cross-tenant and cross-user claims must be demonstrated between **two tenants you control**; never between yours and a real

customer's.

4. **Canary discipline.** Plant uniquely identifiable inert markers - format `GNBB-CANARY-<uuid4>` - so retrieval, leakage, and propagation claims are provable and attributable, and so the target's defenders can separate your traffic from a real adversary's.
5. **Denial-of-wallet is a real harm.** Inference is metered and shared. Stay inside documented rate limits, cap total requests up front, never run volumetric fuzzing, and demonstrate a missing consumption control with the **minimum** request count that shows it - not by actually exhausting the quota, the context window, or the GPU pool.
6. **Stochastic reproducibility.** Model output is nondeterministic; one successful prompt is not a finding. Every AI finding records model and version string, sampling parameters (temperature / top-p / seed where settable), trial count, success count, observed success rate with its confidence interval, and timestamps. **The GreyNOC bar is ≥ 2 successes from a clean context across a stated trial count**, with the full trial ledger in the report - a 2/50 result is reportable as 2/50, never as "reproducible."
7. **Data you will encounter.** RAG corpora, agent memory, vector stores, and inference logs can contain third-party PII, secrets, and conversation content. Capture the minimum proof (a hash, a redacted fragment, a record count), never bulk-collect, never retain, and report immediately. Discovery of real user data is a Section 6 stop condition.
8. **Model artifacts are untrusted code.** Never load, unpickle, deserialize, or execute a model artifact, adapter, notebook, or config obtained from a target. Inspect statically, offline, in an isolated environment.
9. **Agent and tool boundaries.** A test agent must hold the least-privileged tool set that can prove the point, prefer echo/no-op tools for proof, and never chain into out-of-scope systems. You are responsible for every action your agent takes, including ones an injection induced.
10. **Be identifiable.** Where the program permits it, mark traffic (identifying header or user agent) and register test accounts, so defenders do not burn incident-response cycles on you.
11. **Additional stop conditions.** Stop, preserve, and escalate on: evidence another party is exploiting the same defect; any probe that begins to affect other tenants' sessions, outputs, or costs; a training-data or memory extraction that starts returning real user content; or a finding that cannot be proven without generating genuinely harmful content.

8. Defect-class registry and AI evidence notation

8.1 Defect-class prefixes

Every bug-bounty playbook in this library numbers its defect classes with a unique prefix, so a class ID is globally unambiguous in a report, a ticket, or a triage queue. Never reuse a prefix.

Prefix	Range	Owning playbook	Domain
C	C1-C15	PB-08	Cryptographic implementation & PQ migration defects
L	L1-L19	PB-16	LLM application layer
S	S1-S12	PB-17	AI supply chain & model artifacts
G	G1-G14	PB-18	Agentic systems, tools & the MCP boundary
I	I1-I12	PB-20	Inference infrastructure & multi-tenant isolation

A finding cites exactly one primary class. Chains cite the primary class and list the others in sequence (L2 -> L5 -> L6), because the chain is what carries the impact.

Precedence when the one-line heuristics disagree. Each catalog carries a short routing heuristic, and read literally they overlap: PB-16's "delete the model and if a recognizable web or API bug remains it is L" would claim most of I, because a gateway authorization bug survives deleting the model; PB-20's "if it survives replacing the model with a stub that echoes its input it is I" would claim much of L. **The heuristics are aids, not the rule.**

Resolve in this order:

1. **This table wins.** The prefix ranges above are authoritative for ownership. A class that exists in a catalog owns its subject, and no heuristic reassigns it.
2. **Then the layer that makes the decision.** If application code decides, it is L. If the gateway, router, scheduler, cache, or store decides, it is I. If the agent runtime takes an action, it is G. If the defect is in something fetched or loaded, it is S.
3. **Then the scope tables** in each catalog, which agree with each other and are more precise than the one-liners.
4. **Only then the delete-the-model test**, which is a sanity check for whether a finding is an ordinary web bug in new vocabulary - not an ownership rule.

Two consequences worth stating, because both are routinely mis-filed. An OAuth or connector-consent defect reached through an AI feature is G4, not L, even though deleting the model leaves a pure web bug - rule 1 settles it. And a shared artifact **cache** is S, while a shared **serving process** is I.

Four playbooks own no prefix by design. **PB-07** and **PB-15** are methodology, not catalogs - their findings carry the class of whichever catalog the defect belongs to. **PB-19** covers model *behavior*, which routes to a program's model-safety channel rather than its security queue (Section 7.1); giving those findings a class ID would invite a triager to treat them as security defects. **PB-28** is a prompt library for operator tradecraft, not a defect catalog: its P-DEF / P-OFF / P-BTY / P-BUG / P-REP identifiers name *how the analyst worked*, not *what is wrong with the target*, and they are deliberately absent from the table above. A P- identifier is never cited as a finding class - it belongs only in the methodology or AI-assistance-disclosure section of a report (P-REP-07). Note also that PB-28's subject is the assistant *working for the tester*; driving a *target's* model remains PB-15/16/18 scope under Section 7's canary and trial-ledger discipline.

The TX (Transmission & Physical Layer) collection owns no prefix either. TX-01, TX-02 and TX-03 are detection and response playbooks, not bug-bounty catalogs; their intrusion classes are descriptive (*bend-tap*, *rogue-lambda*, *ROADM/monitor*, *impersonation*, *reception-anomaly*) and are cited by name. A registry prefix would invite a triager to read a physical-medium event as an application defect. Note also that C10 in PB-08 covers **computational** side channels - timing, fault, and secret residue. Electromagnetic emanation (TX-01 Section 5.7), optical emission (TX-02 Section 6.7), and the silicon-level physical attacks in TX-03 are different physical phenomena and are deliberately not filed under C10.

Evidence grading in the TX track. TX-03 grades every cited result - [hw] measured on named hardware, [sim] leakage-model or simulation only, [analysis] an analytical or formal result (proof, model, or standards analysis) with no implementation demonstrated, [norm] normative standard text, [adv] advisory standard text, [relayed] the citing authors quote a third party, [vendor] unaudited vendor claim. Any artifact reusing that material carries the grade with it. A [sim] result tabulated beside a [hw] result without its grade visible is a reporting defect, because it overstates demonstrated capability - the same fabrication concern Section 6.3 addresses for findings.

8.2 Canary tokens

Inert markers are the sanctioned proof mechanism for AI findings (Section 7.4). Format:

GNBB-CANARY-<uuid4> e.g. GNBB-CANARY-3f2a1c68-9d47-4e2b-8a10-5c7e9b04d1f6

Canaries are registered in the engagement's canary registry before use, are unique per probe and per tenant, and carry no semantic content. A retrieval, leakage, propagation, or tool-invocation claim is proven by the canary appearing where it should not - not by narrative description.

8.3 Trial-ledger notation for stochastic findings

Per Section 7.6, AI findings report reproducibility as a ledger, never as an adjective. The canonical short form in a report title or triage summary is:

successes/trials @ temperature (95% CI low-high) e.g. 7/20 @ t=1.0 (0.18-0.57)

Report templates may label the interval **95% Wilson CI** where the form needs to be unambiguous; the short form above stays canonical for titles and triage summaries.

The interval is a **Wilson score interval** - with the small trial counts an authorized, rate-limited engagement can afford, a normal-approximation interval is wrong at the tails and a bare percentage is meaningless. $2/50$ is reported as $2/50$, and is still a real finding when the consequence is real and the attacker's retry cost is low; severity comes from consequence and retry economics, not from the rate alone.

9. Versioned reference baseline

These sources are the audit baseline for the collection. Record the retrieval date in any compliance evidence package and re-check status before deployment:

- NIST Post-Quantum Cryptography project - authoritative status for FIPS 203/204/205, FIPS 206 development, HQC selection, and future PQ standards.
- NIST SP 800-227 - KEM construction and implementation guidance.
- NIST CSWP 39-upd1 - final June 2026 crypto-agility guidance.
- IETF ECDHE-MLKEM TLS draft and the IANA TLS Supported Groups registry - current protocol status and code points; an Internet-Draft is not an RFC.
- NIST AI RMF and NIST AI 600-1 GenAI Profile - enterprise AI governance, mapping, measurement, and management baseline.
- NIST CSF 2.0 - Govern, Identify, Protect, Detect, Respond, and Recover outcomes.
- CISA JCDC AI Cybersecurity Collaboration Playbook - AI incident coordination and information-sharing baseline.
- OWASP Top 10 for LLM Applications 2025 and OWASP Top 10 for Agentic Applications 2026 - application and agentic risk cross-checks.
- Executive Order 14412 fact sheet and OMB M-23-02 - federal PQ migration direction; determine applicability before treating either as binding.
- PCI SSC FAQ 1491 - PQ TLS is described as best practice; PCI DSS does not currently impose a general PQ migration mandate.

GreyNOC - detection-engineering-first. Reproducible or it didn't happen.